MACSE502 Programming for Data Science

CAT-2 Coding Cheat Sheet

Parsed from your 10 lab submissions (25 programs) and laid out in the three question formats from the whiteboard photo.

Part 1	193 task rows: Python programs 1 to 17, Scala programs 18 to 25
Part 2	88 debug items with the real error messages (62 Python, 26 Scala)
Part 3	13 fill-in-the-missing-lines programs, 65 blanks with justifications
Part 4	Load-package lines and a function to package index
Part 5	What was verified and what was not

Watch out. The marks and the 10% / 90% split in the photo are my interpretation. Every code row, bug and program in this sheet was run; the exceptions are listed in Part 5.

Contents

How the CAT-2 coding questions look 4

What an answer looks like . . . 4

How this sheet is organised . . . 4

Part 1. Task tables (T.No | Task Desc | Program code) 5

A. Pandas basics: create, select, filter, add, sort, average . . . 5

B. Pandas inspection, missing values, cleaning, scaling . . . 6

C. NumPy arrays: create, statistics, slice, reshape, multiply . . . 6

D. Feature engineering and EDA with Matplotlib / Seaborn . . . 7

E. Web scraping with Beautiful Soup, then CSV / JSON / statistics . . . 8

F. Plotly Express: interactive bar and scatter with trendline . . . 9

G. Bokeh: scatter, multi-line and bar charts . . . 10

H. scikit-learn: StandardScaler, PCA, K-means, silhouette, crosstab . . . 11

Scala tables (programs 18 to 25) . . . 12

S1. Scala collections: arrays, lists, sets, tuples, maps . . . 12

S2. Functional combinators and case classes . . . 13

S3. Saddle: Vec, Index, Series, Frame (Program 23) . . . 13

S4. Breeze vectors and Plotly.Scala bar chart (Program 24) . . . 14

S5. Spire exact arithmetic with a hand-made statistic (Program 25) . . . 15

Part 2. Debug the program (S.No | code | description) 16

2.1 Python: pandas (23 items) . . . 16

2.2 Python: NumPy (10 items) . . . 18

2.3 Python: Matplotlib and seaborn (6 items) . . . 20

2.4 Python: Beautiful Soup and requests (6 items) . . . 20

2.5 Python: Plotly (4 items) . . . 21

2.6 Python: Bokeh (3 items) . . . 22

2.7 Python: scikit-learn: PCA and K-means (6 items) . . . 22

2.8 Python: Plain Python (4 items) . . . 23

2.9 Scala (26 items) . . . 24

Part 3. Fill in the missing lines (Line No | Missing code | Justification) 28

F1. Employee table: create, filter, group, sort (pandas basics) . . . 28

F2. Cleaning a dirty table: duplicates, NaN, text, dates, outliers . . . 29

F3. Reshape, slice and broadcast an array (NumPy) . . . 30

F4. Distribution, scatter and correlation heat-map (matplotlib and seaborn) . . . 30

F5. Scraping job cards with BeautifulSoup . . . 31

F6. Interactive bar chart and OLS trend line (Plotly Express) . . . 32

F7. Iris scatter with colours by species and hover (Bokeh) . . . 32

F8. PCA on the iris data: scale, fit, variance ratio, loadings . . . 33

F9. K-Means on the iris data: elbow, fit, silhouette . . . 34

F10. Arrays, lists, sets and maps . . . 34

F11. Functional combinators and a case class . . . 35
F12. Vectors, statistics and formatted output (Breeze) . . . 36
F13. Series and Frame (Saddle) . . . 36

Part 4. Reference: Load-package rows and function index **38**

4.1 Load package (the first row of almost every Q1 table) . . . 38
4.2 Function to package index . . . 38

Part 5. What was verified and what was not **42**

Verified by running . . . 42
Not verified, or only partly . . . 42

How the CAT-2 coding questions look

This page is my reading of the whiteboard photo in *Prac/Submitions*. The column names and row numbers are what the photo shows. The marks are **my guess**, check them against your own notes.

Format	What the photo shows	What to write	Marks (guess)
1. Task table (Q1)	Columns T.No | Task Desc | Program code. Row 1 is "Load package", row 3 is "Data frame". Each row has a tick, a running count (1, 2, 3) and "+1". "10%" is circled above Task Desc and "90%" above Program code. An "8" is written below the table.	One task per row, in workflow order (load, create, inspect, clean, analyse, plot, save). Code is the exact statements for that task.	+1 per row. The 8 may be the total. 10% / 90% may be column widths or the split of each row's marks (describe the task / write the code), I cannot tell.
2. Debug program	Columns S.No | code | description (the word "code" is written under the last heading, so it may mean "description of the code"). Rows are numbered 1, 2, 4, 5. Row 4 holds a *print( )* line, row 5 is an empty box, dashes fill the description column. A "6" is written below.	Copy the faulty line, then in the description say what is wrong (error message or wrong result) and give the corrected line.	Possibly 6 rows or 6 marks.
3. Missing code (10 Mks)	Columns Line No | Missing code | Justification (underlined). The listed lines are 5, 6, 17, 25 and 27 (boxes are drawn next to the first four), so the program has at least 27 lines.	The exact missing line for each listed number, plus one sentence on why that line is needed.	5 blanks for 10 marks: perhaps 2 each (code and justification).

What an answer looks like

T.No	Task Desc	Program code
1	Load package	`import pandas as pd`
2	Data frame	`df = pd.DataFrame(data)`

S.No	Code	Description
1	`print(df.head)`	Method not called: prints `<bound method ...>`. Use `df.head()`.

Line No	Missing code	Justification
7	`df = pd.DataFrame(data)`	Turns the dict into a table; later lines use df.

How this sheet is organised

- **Part 1** (193 Python and Scala rows in 13 tables): Format 1. A table is one program; the rows are in running order, so you can type them top to bottom.
- **Part 2** (88 bugs): Format 2. 71 stop with an error message, 14 run and print a wrong result, 3 only warn.
- **Part 3** (13 programs, 65 blanked lines): Format 3, with the full program, the justification and the real output.
- **Part 4**: the import line for each library and a function to package index (the 'Function Details' table of every lab record).
- **Part 5**: what was and was not verified. Read it before trusting a number.

Part 1. Task tables (T.No | Task Desc | Program code)

Python programs 1 to 17 (Parts A to H) and Scala programs 18 to 25 (S1 to S5). Rows are written the way the labs solve the tasks, but with small inline data so that every table runs without your CSV files. If the exam gives a file, replace the data-creation row with `pd.read_csv('file.csv')`.

A. Pandas basics: create, select, filter, add, sort, average

Lab records: Programs 1, 2. 21 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Load package	`import pandas as pd`

2	Create the dataset (dictionary)	`data = {'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emma'],`
		`'Age': [23, 30, 28, 35, 22],`
		`'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney']}`

3	Create the DataFrame and display it	`df = pd.DataFrame(data)` `print(df)`
4	First rows, size and column names	`print(df.head(3))` `print(df.shape, df.columns.tolist())`
5	Select only Name and Age columns	`name_age = df[['Name', 'Age']]` `print(name_age)`
6	Filter rows where Age > 25	`filtered_df = df[df['Age'] > 25]` `print(filtered_df)`
7	Filter with two conditions (and / or)	`both = df[(df['Age'] > 25) & (df['City'] != 'Paris')]` `either = df[(df['Age'] > 30) | (df['City'] == 'Sydney')]`
8	Filter with query()	`q = df.query('Age > 25')`
9	Add a column with a default value	`df['Country'] = 'India'` `print(df)`
10	Add a derived column	`df['Age_Next_Year'] = df['Age'] + 1`
11	Sort by Age in descending order	`sorted_df = df.sort_values(by='Age', ascending=False)` `print(sorted_df)`
12	Calculate the average Age	`average_age = df['Age'].mean()` `print('Average Age =', average_age)`
13	Other statistics of a column	`print(df['Age'].max(), df['Age'].min(), df['Age'].median(), df['Age'].sum())`
14	Top 3 rows by Age	`top3 = df.sort_values('Age', ascending=False).head(3)`
15	Drop a column	`slim = df.drop(columns=['Age_Next_Year'])`

16	Product table: 20 ids with a list comprehension	`amounts = [45000, 62000, 78000, 35000, 91000, 54000, 28000,`
		`67000, 83000, 41000, 56000, 72000, 39000, 88000,`
		`47000, 61000, 95000, 33000, 70000, 58000]`
		`sales = pd.DataFrame({'ProductID': [f'PRD{i}' for i in range(1, 21)],`
		`'SalesAmount': amounts,`
		`'Category': ['Electronics', 'Clothing', 'Electronics',`
		`'Grocery', 'Home'] * 4})`

17	Filter SalesAmount > 50000 and add Region	`big = sales[sales['SalesAmount'] > 50000].copy()` `big['Region'] = 'South India'`

18	Group by category: total, sorted	`cat_sales =`
		`sales.groupby('Category')['SalesAmount'].sum().sort_values(ascending=False)`
		`print(cat_sales)`

19	Best and weakest category	`print(cat_sales.idxmax(), cat_sales.max(), cat_sales.idxmin(), cat_sales.min())`
20	Count rows per category	`print(sales['Category'].value_counts())`
21	Export to CSV and JSON	`sales.to_csv('sales.csv', index=False)` `sales.to_json('sales.json', orient='records', indent=2)`

B. Pandas inspection, missing values, cleaning, scaling

Lab records: Programs 3, 4. 16 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Load packages	`import pandas as pd` `import numpy as np`
2	DataFrame from nested lists	`stud = [['Jhon', 20, 85], ['Mary', 21, 90], ['David', 19, 78]]` `df0 = pd.DataFrame(stud, columns=['Name', 'Age', 'Marks'])`

3	Core attributes	`print(df0.size, df0.shape, df0.ndim)`
		`print(df0.columns, df0.index)`
		`print(df0.dtypes)`

4	Select one column / several columns	`print(df0['Name'])` `print(df0[['Name', 'Age']])`
5	loc (labels) and iloc (positions)	`print(df0.loc[0, 'Name'], df0.iloc[0:2, 0:2])`

6	Dataset with missing values	`data = {'Name': ['Alice', 'Bob', np.nan, 'David', 'Emma'],`
		`'Age': [23.0, np.nan, 28.0, 35.0, np.nan],`
		`'City': ['New York', 'London', 'Paris', np.nan, 'Sydney']}`
		`df = pd.DataFrame(data)`

7	Detect missing values	`print(df.isnull())`
		`print(df.isnull().sum())`
		`print(df.notnull())`

8	Preview: fill every NaN with 0	`fill_0 = df.fillna(0)`

9	Fill Age with the mean age	`mean_age = df['Age'].mean()`
		`df['Age'] = df['Age'].fillna(mean_age)`
		`print(df)`

10	Drop rows where Name or City is missing	`df = df.dropna(subset=['Name', 'City'])` `print(df)`
11	Convert Age to integer	`df['Age'] = df['Age'].astype(int)` `print(df.dtypes)`

12	Min-max scaling of Age	`df['Norm_Age'] = (df['Age'] - df['Age'].min()) / (df['Age'].max() -`
		`df['Age'].min())`
		`print(df)`

13	Remove duplicates, reset the index	`df = df.drop_duplicates().reset_index(drop=True)`
14	Rename a column, clean text	`df = df.rename(columns={'Norm_Age': 'Age_Scaled'})` `df['Name'] = df['Name'].str.strip().str.title()`

15	Fleet example: fill with mean then drop	`fleet = pd.DataFrame({'VehicleID': ['V101', 'V102', 'V103'],`
		`'FuelConsumption': [15.0, np.nan, 22.0], 'Route': ['R1', 'R2', np.nan]})`
		`fleet['FuelConsumption'] =`
		`fleet['FuelConsumption'].fillna(fleet['FuelConsumption'].mean())`
		`fleet = fleet.dropna(subset=['VehicleID', 'Route'])`

16	Flag with np.where / group report	`fleet['Level'] = np.where(fleet['FuelConsumption'] >= 18, 'High', 'Low')`
		`report = fleet.groupby('Route')['FuelConsumption'].agg(['sum',`
		`'mean']).reset_index()`

Note. Order matters: fillna first, then dropna, then astype(int) (an int column cannot hold NaN), then scaling.

C. NumPy arrays: create, statistics, slice, reshape, multiply

Lab records: Programs 5, 6. 16 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Load package	`import numpy as np`

2	Random integers 0..10 in a (3, 4) array	`np.random.seed(0)`
		`arr = np.random.randint(0, 11, size=(3, 4))`
		`print(arr)`

3	Shape, size, dtype	`print(arr.shape, arr.size, arr.dtype)`
4	Mean and standard deviation of the whole array	`print(np.mean(arr))` `print(np.std(arr))`
5	First two rows and last two columns	`sub = arr[:2, -2:]` `print(sub)`
6	Reshape to (4, 3)	`reshaped = arr.reshape(4, 3)` `print(reshaped)`

7	Element-wise multiplication with another (4, 3) array	`other = np.random.randint(1, 5, size=(4, 3))`
		`product = reshaped * other`
		`print(product)`

8	Explicit 4 x 5 array	`production = np.array([[120, 135, 128, 110, 140],`
		`[95, 105, 100, 90, 98],`
		`[150, 145, 160, 155, 148],`
		`[80, 85, 78, 82, 75]])`

T.No	Task Desc	Program code
9	Slice first 3 rows and last 2 columns	`print(production[:3, -2:])`
10	Reshape (4, 5) to (5, 4) and multiply by a weight matrix	`w = np.full((5, 4), 0.9)` `effective = production.reshape(5, 4) * w`
11	Row-wise and column-wise statistics	`print(np.mean(production, axis=1))` `print(np.sum(production, axis=0))`

12	Rank rows by total (highest first)	`totals = np.sum(production, axis=1)`
		`ranking = np.argsort(totals)[::-1]`
		`print(ranking)`

13	Shifts below the overall average	`shift_avg = np.mean(production, axis=0)` `low = np.where(shift_avg < shift_avg.mean())[0]`
14	Label values with np.where	`labels = np.where(totals >= totals.mean(), 'High', 'Low')`
15	Matrix product and transpose	`m = production @ production.T` `print(m.shape, production.T.shape)`
16	Useful creators	`print(np.arange(0, 10, 2), np.linspace(0, 1, 5), np.zeros((2, 2)), np.ones(3))`

Note. np.std divides by n (population). For the sample standard deviation use np.std(a, ddof=1). randint(0, 11) gives 0..10 because the upper bound is excluded.

D. Feature engineering and EDA with Matplotlib / Seaborn

Lab records: Programs 7, 8. 14 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code

1	Load packages	`import numpy as np`
		`import pandas as pd`
		`import matplotlib`
		`matplotlib.use('Agg')`
		`import matplotlib.pyplot as plt`
		`import seaborn as sns`

2	Create random data	`np.random.seed(42)`
		`df = pd.DataFrame({'Feature1': np.random.rand(50) * 100 + 1,`
		`'Feature2': np.random.randn(50) * 15 + 50,`
		`'Target': np.random.randint(0, 2, 50)})`

3	Logarithm of Feature1	`df['Log_Feature1'] = np.log(df['Feature1'])`

4	Bin Feature2 into low / medium / high	`df['Feature2_Bin'] = pd.cut(df['Feature2'], bins=3, labels=['low', 'medium',`
		`'high'])`
		`print(df['Feature2_Bin'].value_counts())`

5	Bin with your own limits	`df['Band'] = pd.cut(df['Feature2'], bins=[0, 45, 55, np.inf],` `labels=['Optimal', 'Moderate', 'Degraded'], right=False)`

6	Bin with an if / elif function	`def categorize(v):`
		`if v < 45:`
		`return 'Normal'`
		`elif v < 55:`
		`return 'Elevated'`
		`else:`
		`return 'Hypertensive'`
		`df['Level'] = df['Feature2'].apply(categorize)`

7	Correlation matrix	`corr = df[['Feature1', 'Feature2', 'Target', 'Log_Feature1']].corr()` `print(corr)`

8	Scatter plot coloured by Target	`plt.figure(figsize=(6, 4))`
		`plt.scatter(df['Feature1'], df['Feature2'], c=df['Target'], cmap='viridis')`
		`plt.colorbar(label='Target')`
		`plt.xlabel('Feature1')`
		`plt.ylabel('Feature2')`
		`plt.title('Feature1 vs Feature2 by Target')`

9	Save the figure, then show it	`plt.savefig('scatter.png', dpi=120)` `plt.show()`
10	Seaborn scatter with hue	`plt.figure()` `sns.scatterplot(x='Feature1', y='Feature2', hue='Target', data=df)`
11	Box plot of a value grouped by category	`plt.figure()` `sns.boxplot(x='Target', y='Feature2', data=df)`
12	Heatmap of the correlation matrix	`plt.figure()` `sns.heatmap(corr, annot=True, cmap='coolwarm')`

13	Two plots side by side	`fig, axes = plt.subplots(1, 2, figsize=(10, 4))`
		`axes[0].hist(df['Feature2'], bins=8)`
		`axes[0].set_title('Feature2')`
		`axes[1].bar(['low', 'medium', 'high'],`
		`df['Feature2_Bin'].value_counts().reindex(['low', 'medium', 'high']))`
		`axes[1].set_title('Bins')`
		`plt.tight_layout()`

T.No	Task Desc	Program code
14	Interpretation sentence	`# Feature1 and Log_Feature1 correlate strongly (the log is a monotone transform);` `# correlation of Feature2 with Target is near 0, so the groups overlap in the scatter plot.`

Note. Call plt.savefig() BEFORE plt.show(): after show() the figure is closed and the saved file is blank.

E. Web scraping with Beautiful Soup, then CSV / JSON / statistics

Lab records: Programs 9, 10. 15 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code

1	Load packages	`import requests`
		`from bs4 import BeautifulSoup`
		`import pandas as pd`
		`import json`

2	Fetch a live page (guarded) *(not run: needs internet)*	`resp = requests.get('https://quotes.toscrape.com', timeout=3)` `html = resp.text if resp.status_code == 200 else ''`

3	Offline HTML fallback used in the lab	`html_doc = """`
		`<div class='quote'>`
		`  <span class='text'>"Stay hungry, stay foolish."</span>`
		`  <small class='author'>Steve Jobs</small>`
		`  <div class='tags'><a class='tag'>inspirational</a><a class='tag'>wisdom</a></div>`
		`</div>`
		`<div class='quote'>`
		`  <span class='text'>"Knowledge is power."</span>`
		`  <small class='author'>Francis Bacon</small>`
		`  <div class='tags'><a class='tag'>knowledge</a><a class='tag'>wisdom</a></div>`
		`</div>`
		`<div class='quote'>`
		`  <span class='text'>"Imagination is more important than knowledge."</span>`
		`  <small class='author'>Steve Jobs</small>`
		`  <div class='tags'><a class='tag'>imagination</a></div>`
		`</div>`
		`"""`

4	Parse the HTML	`soup = BeautifulSoup(html_doc, 'html.parser')`
5	Find all repeated blocks	`cards = soup.find_all('div', class_='quote')` `print(len(cards))`

6	Extract text, author and tags of each block	`records = []`
		`for card in cards:`
		`  text_elem = card.find('span', class_='text')`
		`  author_elem = card.find('small', class_='author')`
		`  tags = [t.get_text().strip() for t in card.find_all('a', class_='tag')]`
		`  quote = text_elem.get_text().strip('"') if text_elem else 'N/A'`
		`  author = author_elem.get_text().strip() if author_elem else 'Unknown'`
		`  records.append({'Quote': quote, 'Author': author, 'Tags': ', '.join(tags), 'Tag_Count': len(tags), 'Quote_Length': len(quote)})`

7	Build the DataFrame	`scraped_df = pd.DataFrame(records)` `print(scraped_df.head())`
8	Export to CSV and JSON	`scraped_df.to_csv('quotes.csv', index=False)` `scraped_df.to_json('quotes.json', orient='records', indent=2)`
9	Author frequency	`author_counts = scraped_df['Author'].value_counts()` `print(author_counts)`
10	Descriptive statistics	`print(scraped_df[['Quote_Length', 'Tag_Count']].describe().round(2))`
11	Most frequent tags	`all_tags = [t.strip() for s in scraped_df['Tags'] for t in s.split(',') if t.strip()]` `print(pd.Series(all_tags).value_counts().head(5))`
12	CSS-selector alternative	`texts = [e.get_text() for e in soup.select('div.quote span.text')]`

13	Job cards: number parsed from text	`job_html = """`
		`<div class='job-card'><h2 class='title'>Data Engineer</h2>`
		`  <span class='loc'>Pune</span><span class='sal'>26 LPA</span></div>`
		`<div class='job-card'><h2 class='title'>ML Engineer</h2>`
		`  <span class='loc'>Pune</span><span class='sal'>22 LPA</span></div>`
		`<div class='job-card'><h2 class='title'>DevOps</h2>`
		`  <span class='loc'>Chennai</span><span class='sal'>18 LPA</span></div>`
		`"""`
		`jobs = []`
		`for c in BeautifulSoup(job_html, 'html.parser').find_all('div', class_='job-card'):`
		`  salary = float(c.find('span', class_='sal').text.replace(' LPA', '').strip())`
		`  jobs.append({'Job_Title': c.find('h2', class_='title').text,`
		`  'Location': c.find('span', class_='loc').text,`
		`  'Salary_LPA': salary})`
		`job_df = pd.DataFrame(jobs)`

T.No	Task Desc	Program code
14	Group and aggregate (named aggregation)	`loc_stats = job_df.groupby('Location').agg(Job_Count=('Job_Title', 'count'), Average_Salary_LPA=('Salary_LPA', 'mean')).sort_values(by='Job_Count', ascending=False)` `print(loc_stats)`

15	Bar chart of the counts	`import matplotlib`
		`matplotlib.use('Agg')`
		`import matplotlib.pyplot as plt`
		`plt.figure()`
		`plt.bar(loc_stats.index, loc_stats['Job_Count'], color='steelblue', edgecolor='black')`
		`plt.title('Openings by city')`
		`plt.xlabel('City')`
		`plt.ylabel('Count')`
		`plt.savefig('jobs.png', dpi=120)`

Note. find() returns the first match (or None); find_all() returns a list. class is a Python keyword, so the argument is class_.

F. Plotly Express: interactive bar and scatter with trendline

Lab records: Programs 11, 12. 12 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code

1	Load packages	`import numpy as np`
		`import pandas as pd`
		`import plotly.express as px`

2	Dataset and derived indicators	`df = pd.DataFrame({'IT_Hub': ['Bengaluru', 'Hyderabad', 'Pune', 'Chennai'], 'Workforce_K': [1650.0, 1150.0, 820.0, 780.0], 'Export_Billion_USD': [78.5, 52.0, 36.2, 34.0]})`
		`df['National_Share_Pct'] = (df['Workforce_K'] / df['Workforce_K'].sum() * 100).round(2)`
		`df['Export_Per_Worker_USD'] = (df['Export_Billion_USD'] * 1e9 / (df['Workforce_K'] * 1000)).round(2)`
		`df = df.sort_values(by='Workforce_K', ascending=False).reset_index(drop=True)`

3	Interactive bar chart	`fig = px.bar(df, x='IT_Hub', y='Workforce_K', color='Workforce_K', color_continuous_scale='Tealgrn', text='Workforce_K', title='Tech workforce by hub', labels={'Workforce_K': 'Workforce (thousands)', 'IT_Hub': 'Hub'})`
4	Value labels and hover template	`fig.update_traces(texttemplate='%{text:.0f}k', textposition='outside', hovertemplate='<b>%{x}</b> Workforce: %{y:,.1f}k<extra></extra>')`
5	Layout: titles, axis range, background	`fig.update_layout(title_font=dict(size=18), xaxis=dict(tickangle=-25), yaxis=dict(range=[0, 1900]), plot_bgcolor='rgba(245,248,250,1)', margin=dict(l=60, r=40, t=70, b=80))`
6	Show on screen / save as HTML	`fig.show()` `fig.write_html('hubs.html')`
7	Save as PNG (needs kaleido) *(not run: kaleido is not installed here)*	`fig.write_image('hubs.png', width=750, height=450, scale=2)`

8	Scatter data and correlation	`np.random.seed(42)`
		`area = np.sort(np.random.uniform(15000, 120000, 30))`
		`lease = area * np.random.uniform(1100, 1550, 30) / 100000.0 + np.random.normal(0, 15, 30)`
		`prop = pd.DataFrame({'Floor_Area_SqFt': area.round(0), 'Annual_Lease_Lakhs': lease.round(2), 'Occupancy_Pct': np.clip(np.random.normal(88, 7, 30), 65, 99).round(1)})`
		`print(prop['Floor_Area_SqFt'].corr(prop['Annual_Lease_Lakhs']))`

9	Least-squares line (slope, intercept)	`slope, intercept = np.polyfit(prop['Floor_Area_SqFt'], prop['Annual_Lease_Lakhs'], 1)` `print(slope, intercept)`
10	Scatter with size, colour scale and OLS trendline	`fig2 = px.scatter(prop, x='Floor_Area_SqFt', y='Annual_Lease_Lakhs', size='Occupancy_Pct', color='Annual_Lease_Lakhs', color_continuous_scale='Plasma', trendline='ols', trendline_color_override='#222222', title='Area vs lease')`
11	Hover text from other columns	`fig2.update_traces(marker=dict(opacity=0.88, line=dict(width=1.5, color='black')), selector=dict(mode='markers'))` `fig2.update_layout(coloraxis_colorbar=dict(title='Lease', thickness=15, len=0.75))`
12	Annotation text on the chart	`fig2.add_annotation(x=70000, y=350, text='Linear fit', showarrow=False)` `fig2.write_html('lease.html')`

Note. trendline='ols' needs statsmodels; write_image needs kaleido (>=1) installed.

G. Bokeh: scatter, multi-line and bar charts

Lab records: Programs 13, 14, 15. 17 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code

1	Load packages	`import pandas as pd`
		`from sklearn.datasets import load_iris`
		`from bokeh.plotting import figure, show, output_file, save`
		`from bokeh.models import ColumnDataSource, HoverTool, LabelSet`
		`from bokeh.transform import factor_cmap`

2	Iris data frame with species names	`raw = load_iris()`
		`df = pd.DataFrame(raw.data, columns=['sepal_length', 'sepal_width',`
		`    'petal_length', 'petal_width'])`
		`names = ['Setosa', 'Versicolor', 'Virginica']`
		`df['species'] = [names[i] for i in raw.target]`

3	Wrap the data in a ColumnDataSource	`source = ColumnDataSource(df)`
4	Colour map by category	`color_map = factor_cmap('species', palette=['#D95F02', '#7570B3', '#1B9E77'],` `    factors=names)`
5	Hover tool (fields start with @)	`hover = HoverTool(tooltips=[('Species', '@species'), ('Petal Length',` `    '@petal_length{0.1f} cm'), ('Sepal Length', '@sepal_length{0.1f} cm')])`

6	Create the figure	`p = figure(title='Petal vs sepal length', x_axis_label='Petal Length (cm)',`
		`    y_axis_label='Sepal Length (cm)',`
		`           width=720, height=440, tools=[hover, 'pan', 'wheel_zoom',`
		`               'box_zoom', 'reset', 'save'])`

7	Scatter glyph with legend	`p.scatter('petal_length', 'sepal_length', source=source, color=color_map,`
		`    size=11, alpha=0.82, legend_field='species')`
		`p.legend.location = 'top_left'`

8	Output file and show	`output_file('iris_bokeh.html')` `show(p)`

9	Multi-line data	`days = [f'Day {i}' for i in range(1, 8)]`
		`net = pd.DataFrame({'Day': days, 'US_East_ms': [45.2, 48.1, 42.6, 51.3, 47.0,`
		`    44.5, 49.2],`
		`                   'AP_South_ms': [28.5, 31.2, 29.8, 36.4, 33.1, 30.2, 34.7],`
		`                       'SLA_Limit_ms': [100.0] * 7})`
		`src2 = ColumnDataSource(net)`

10	Line figure with a categorical x axis	`p2 = figure(title='Latency', x_range=days, width=720, height=440, tools=['pan',` `    'wheel_zoom', 'reset', 'save'])`

11	One line per series, with markers	`p2.line('Day', 'US_East_ms', source=src2, legend_label='US-East', line_width=3,`
		`    color='#386CB0')`
		`p2.scatter('Day', 'US_East_ms', source=src2, size=8, color='#386CB0',`
		`    fill_color='white')`
		`p2.line('Day', 'AP_South_ms', source=src2, legend_label='AP-South',`
		`    line_width=3, color='#33A02C')`

12	Dashed threshold line	`p2.line('Day', 'SLA_Limit_ms', source=src2, legend_label='SLA 100 ms',` `    line_width=2, color='#E41A1C', line_dash='dashed')`
13	Clickable legend	`p2.legend.location = 'top_left'` `p2.legend.click_policy = 'hide'`

14	Bar chart data	`cost = pd.DataFrame({'Department': ['AI Research', 'Engineering',`
		`    'Data Platform'], 'Actual_Spend_USD_K': [485.0, 365.0, 280.0]})`
		`cost['Spend_Label'] = [f'${int(a)}k' for a in cost['Actual_Spend_USD_K']]`
		`src3 = ColumnDataSource(cost)`
		`dept = cost['Department'].tolist()`

15	Bar chart (vbar) with a categorical axis	`p3 = figure(x_range=dept, title='Spend by department', width=720, height=440)`
		`p3.vbar(x='Department', top='Actual_Spend_USD_K', width=0.62, source=src3,`
		`       color=factor_cmap('Department', palette=['#e41a1c', '#377eb8',`
		`           '#4daf4a'], factors=dept), legend_field='Department')`

16	Numbers on top of the bars (LabelSet)	`labels = LabelSet(x='Department', y='Actual_Spend_USD_K', text='Spend_Label',`
		`    level='glyph', x_offset=-16, y_offset=4, source=src3)`
		`p3.add_layout(labels)`
		`p3.xaxis.major_label_orientation = 0.35`

17	Save to HTML / export image	`save(p3, filename='bar_bokeh.html')` `# export_png(p3, filename='bar.png')   needs selenium + a browser driver`

Note. Bokeh 3 uses p.scatter(...) for markers and legend_field= / legend_label=; the old legend= argument was removed.

H. scikit-learn: StandardScaler, PCA, K-means, silhouette, crosstab

Lab records: Programs 16, 17. 15 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code

1	Load packages	`import pandas as pd`
		`from sklearn.datasets import load_iris`
		`from sklearn.preprocessing import StandardScaler`
		`from sklearn.decomposition import PCA`
		`from sklearn.cluster import KMeans`
		`from sklearn.metrics import silhouette_score`
		`import plotly.express as px`

2	Load Iris, features and labels	`iris = load_iris()`
		`X = iris.data`
		`y = iris.target`
		`feature_names = ['Sepal_Length', 'Sepal_Width', 'Petal_Length', 'Petal_Width']`
		`species_names = ['Setosa', 'Versicolor', 'Virginica']`

3	Standardise the features	`scaler = StandardScaler()` `X_scaled = scaler.fit_transform(X)`

4	Fit PCA on all components: variance ratios	`pca_full = PCA(n_components=4)`
		`pca_full.fit(X_scaled)`
		`var_ratios = pca_full.explained_variance_ratio_`
		`cum_var = var_ratios.cumsum()`
		`print(var_ratios.round(4), cum_var.round(4))`

5	Scree table	`scree_df = pd.DataFrame({'Principal_Component': ['PC1', 'PC2', 'PC3', 'PC4'],`
		`'Explained_Variance_Pct': (var_ratios * 100).round(2),`
		`'Cumulative_Variance_Pct': (cum_var * 100).round(2)})`
		`print(scree_df.to_string(index=False))`

6	Reduce to 2 components	`pca_2d = PCA(n_components=2)`
		`X_pca = pca_2d.fit_transform(X_scaled)`
		`pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])`
		`pca_df['Species'] = [species_names[i] for i in y]`

7	Component loadings (eigenvectors)	`loadings = pd.DataFrame(pca_2d.components_.T, columns=['PC1_Loading',`
		`'PC2_Loading'], index=feature_names).round(4)`
		`print(loadings)`

8	Interactive 2-D scatter of the components	`fig = px.scatter(pca_df, x='PC1', y='PC2', color='Species', symbol='Species',`
		`labels={'PC1': f'PC1 ({var_ratios[0] * 100:.2f}%)', 'PC2':`
		`f'PC2 ({var_ratios[1] * 100:.2f}%)'})`
		`fig.show()`

9	Inertia and silhouette for k = 2..6	`k_vals = list(range(2, 7))`
		`inertias, sils = [], []`
		`for k in k_vals:`
		`km = KMeans(n_clusters=k, random_state=42, n_init=10)`
		`labels = km.fit_predict(X_scaled)`
		`inertias.append(round(km.inertia_, 2))`
		`sils.append(round(silhouette_score(X_scaled, labels), 4))`
		`print(pd.DataFrame({'k': k_vals, 'Inertia': inertias, 'Silhouette': sils}))`

10	Final model with k = 3	`kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)`
		`cluster_labels = kmeans.fit_predict(X_scaled)`
		`print(kmeans.inertia_, silhouette_score(X_scaled, cluster_labels))`

11	Cluster table with true species	`k3_df = pd.DataFrame(X, columns=feature_names)`
		`k3_df['Cluster'] = [f'Cluster {c}' for c in cluster_labels]`
		`k3_df['True_Species'] = [species_names[i] for i in y]`

12	Contingency matrix: clusters vs real species	`ct = pd.crosstab(k3_df['True_Species'], k3_df['Cluster'], margins=True)` `print(ct)`

13	Centroids back in original units	`centroids = pd.DataFrame(scaler.inverse_transform(kmeans.cluster_centers_),`
		`columns=feature_names)`
		`print(centroids.round(2))`

14	Interactive 3-D scatter	`fig3 = px.scatter_3d(k3_df, x='Sepal_Length', y='Petal_Length',`
		`z='Petal_Width', color='Cluster')`
		`fig3.show()`

15	Save the plot	`fig3.write_html('kmeans3d.html')`

Note. fit_transform on training data only; kmeans.labels_ and pca.explained_variance_ratio_ end with an underscore (learned attributes).

Scala tables (programs 18 to 25)

Statements marked as the entry point go inside `object Experiment { def main(args: Array[String]): Unit = { ... } }`. *Each table was assembled into one program, compiled with Scala 2.13.12 and run.*

S1. Scala collections: arrays, lists, sets, tuples, maps

Lab records: Programs 18, 19, 20. 21 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Import for mutable collections	`import scala.collection.mutable`

2	Entry point of every lab program	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

3	Array of integers	`val intArr: Array[Int] = Array(2, 5, 9, 14, 20)`

4	Function that returns the sum	`def sumArray(arr: Array[Int]): Int = arr.sum`
		`val totalSum = sumArray(intArr)`
		`println(s"Sum of integer array elements: $totalSum")`

5	Array of five fruit names	`val fruitArr: Array[String] = Array("Apple", "Banana", "Cherry", "Date",` `  "Elderberry")`
6	Function that joins them with commas	`def concatenateFruits(arr: Array[String]): String = arr.mkString(", ")` `println(concatenateFruits(fruitArr))`

7	Iterate and print each element	`for (x <- intArr) println(x)`
		`intArr.foreach(println)`
		`intArr.zipWithIndex.foreach { case (e, i) =>`
		`  println(s"Element at index $i: $e") }`

8	Access, update and length (round brackets!)	`println(intArr(0))`
		`intArr(0) = 99`
		`println(intArr.length)`

9	List of the first five primes	`val primes: List[Int] = List(2, 3, 5, 7, 11)`
10	Function that squares every element	`def squareList(lst: List[Int]): List[Int] = lst.map(x => x * x)` `println(squareList(primes))`
11	Set of unique characters of a string	`val charSet1: Set[Char] = "hello world".toSet` `val charSet2: Set[Char] = "world wide web".toSet`
12	Function returning the intersection of two sets	`def setIntersection[T](s1: Set[T], s2: Set[T]): Set[T] = s1 & s2` `println(setIntersection(charSet1, charSet2))`
13	Union and difference	`println(charSet1 | charSet2)` `println(charSet1 -- charSet2)`

14	Tuple of Int, String, Boolean	`val rec: (Int, String, Boolean) = (260020, "Rohan Yadav", true)`
		`println(rec._1)`
		`println(rec._2)`
		`println(rec._3)`

15	Destructure a tuple	`val (regId, name, active) = rec`
16	Two maps of grades	`val map1 = Map("Rohan" -> 92, "Vedant" -> 88, "Priya" -> 84)` `val map2 = Map("Rohan" -> 85, "Vedant" -> 95, "Kalyan" -> 90)`

17	Merge two maps, keep the higher grade	`def mergeResolveMax(m1: Map[String, Int], m2: Map[String, Int]): Map[String,`
		`  Int] =`
		`  (m1.keySet ++ m2.keySet).map { k =>`
		`  k -> Math.max(m1.getOrElse(k, 0), m2.getOrElse(k, 0))`
		`  }.toMap`
		`val merged = mergeResolveMax(map1, map2)`

18	Print a map sorted by key	`merged.toSeq.sortBy(_._1).foreach { case (k, v) => println(s"  - $k -> $v") }`

19	Mutable HashMap: a duplicate key overwrites	`val grades = mutable.HashMap("Alice" -> 85, "Bob" -> 78)`
		`grades.put("Alice", 95)`
		`grades("Carol") = 70`

20	Collision chaining: a list of values per key	`val multiGradeMap = mutable.Map[String, List[Int]]()`
		`def insertChained(name: String, grade: Int): Unit = {`
		`  val existing = multiGradeMap.getOrElse(name, Nil)`
		`  multiGradeMap(name) = existing :+ grade`
		`}`
		`insertChained("Vedant", 88)`
		`insertChained("Vedant", 95)`
		`println(multiGradeMap)`

21	Look up safely	`println(map1.get("Rohan"))`
		`println(map1.getOrElse("Zed", 0))`
		`println(map1.contains("Priya"))`

Note. Arrays use (i) not [i]; Array/Map/Set literals need no new. A mutable collection needs the import; the default Map and Set are immutable.

S2. Functional combinators and case classes

Lab records: Programs 21, 22. 14 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Case class (defined outside the object)	`case class Person(name: String, age: Int, city: String)`

2	Entry point	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

3	List of integers 1 to 10	`val nums: List[Int] = (1 to 10).toList`
4	filter: keep the even numbers	`val evens = nums.filter(x => x % 2 == 0)`
5	map: multiply every element by 3	`val tripled = nums.map(x => x * 3)`
6	flatMap: pair each integer with its square	`val pairedWithSquare = nums.flatMap(x => List((x, x * x)))`
7	foldLeft: product of all elements	`val totalProduct = nums.foldLeft(1)((acc, elem) => acc * elem)` `println(totalProduct)`

8	More combinators	`println(nums.reduce(_ + _))`
		`println(nums.sum)`
		`println(nums.exists(_ > 9))`
		`println(nums.take(3))`
		`println(nums.partition(_ % 2 == 0))`

9	List of Person objects	`val people = List(`
		`  Person("Vedant Nimkar", 24, "Mumbai"),`
		`  Person("Ayush Sharma", 32, "Delhi"),`
		`  Person("Rohan Yadav", 28, "Bengaluru"),`
		`  Person("Priya Patel", 35, "Mumbai"),`
		`  Person("Sneha Reddy", 22, "Hyderabad"))`

10	Filter people older than 30	`val olderThan30 = people.filter(_.age > 30)` `olderThan30.foreach(p => println(s"  - ${p.name} (Age: ${p.age})"))`

11	Group people by city	`val groupedByCity = people.groupBy(_.city)`
		`groupedByCity.foreach { case (city, group) =>`
		`  println(s"  - $city: ${group.map(_.name).mkString(", ")}") }`

12	Turn each Person into a sentence	`val formatted = people.map(p =>`
		`  s"${p.name} is ${p.age} years old and lives in ${p.city}")`
		`formatted.foreach(println)`

13	Sort by age (ascending / descending)	`val sortedByAge = people.sortBy(_.age)`
		`val oldestFirst = people.sortBy(-_.age)`
		`sortedByAge.zipWithIndex.foreach { case (p, idx) =>`
		`  println(s"[Rank ${idx + 1}] ${p.name} (Age ${p.age}, ${p.city})") }`

14	copy, equality and pattern matching	`val older = people.head.copy(age = 25)`
		`println(Person("A", 1, "X") == Person("A", 1, "X"))`
		`println(people.head match { case Person(n, a, _) if a > 18 =>`
		`  s"$n is an adult"; case _ => "minor" })`

Note. Case-class fields are public vals: p.age works, p.age = 30 does not compile. Use copy to change a field.

S3. Saddle: Vec, Index, Series, Frame (Program 23)

Lab records: Program 23. 12 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Import Saddle	`import org.saddle._` `import org.saddle.ops.BinOps._`

2	Entry point	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

3	Vec of values and an Index of labels	`val prices = Vec(120.50, 125.00, 118.75, 130.25, 128.00)`
		`val quantities = Vec(45.0, 50.0, 42.0, 55.0, 48.0)`
		`val dates = Index("2026-05-01", "2026-05-08", "2026-05-15", "2026-05-22",`
		`  "2026-05-29")`

4	Series = values + index	`val applePrice = Series(prices, dates)`
5	Average price	`val avgPrice = applePrice.values.mean` `println(f"Average Price of Apples: INR $avgPrice%.2f")`
6	Frame of aligned columns	`val frame = Frame("Price" -> prices, "Qty" -> quantities)` `println(frame.numRows + " x " + frame.numCols)`
7	Take one column as a Series	`val priceCol = frame.firstCol("Price")`

T.No	Task Desc	Program code
8	Filter by a condition on values	`val expensive = applePrice.filter(_ > 125.0)` `println(expensive.index.toVec)`

9	Filter rows with a Boolean mask (fruit == Apple)	`val fruit = Vec("Apple", "Apple", "Banana", "Apple", "Orange")`
		`val mask = fruit.map(_ == "Apple")`
		`val applePrices = prices.take(mask.find(identity).toArray)`
		`println(applePrices.mean)`

10	Column arithmetic	`val revenue = frame.firstCol("Price").toVec * frame.firstCol("Qty").toVec` `println(revenue)`

11	Missing values (NaN) are skipped by statistics	`val withNA = Vec(2.0, Double.NaN, 4.0)`
		`println(withNA.mean)`
		`println(withNA.count)`

12	Vegas line chart (needs the Vegas library)
	(not run: no Vegas build for Scala 2.13)
		`import vegas._`
		`val plot = Vegas("Apple Price Trend Over Time")`
		`.withData(Seq(Map("date" -> "2026-05-01", "price" -> 120.5), Map("date" -> "2026-05-08", "price" -> 125.0)))`
		`.mark(Line)`
		`.encodeX("date", Temporal)`
		`.encodeY("price", Quantitative)`

Note. Saddle: Series(values, index). The original org.scala-saddle is not published for Scala 2.13 and Vegas has no Scala 2.13 build; these were verified with the maintained fork io.github.pityka (4.0.0-M14).

S4. Breeze vectors and Plotly.Scala bar chart (Program 24)

Lab records: Program 24. 13 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Import Breeze	`import breeze.linalg._` `import breeze.stats._`

2	Entry point	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

3	Names, prices and quantities as DenseVectors	`val fruits = Array("Apple", "Banana", "Orange", "Mango", "Grape")`
		`val prices = DenseVector(120.0, 45.0, 80.0, 150.0, 95.0)`
		`val quantities = DenseVector(250.0, 600.0, 320.0, 180.0, 210.0)`

4	Revenue = price x quantity (element-wise *:*)	`val revenues = prices *:* quantities`
5	Total and mean	`val totalGross = sum(revenues)` `val meanRevenue = mean(revenues)`

6	Formatted output per fruit	`for (i <- 0 until fruits.length) {`
		`  println(f"  ${fruits(i)}%-7s -> INR ${revenues(i)}%,10.2f")`
		`}`
		`println(f"Total Gross Revenue: INR $totalGross%,10.2f")`

7	Dot product, scalar multiplication, addition	`println(prices dot quantities)`
		`println(prices * 2.0)`
		`println(prices + quantities)`

8	Spread: standard deviation and variance	`println(stddev(prices))` `println(variance(prices))`

9	DenseMatrix, transpose, product, solve	`val m = DenseMatrix((2.0, 1.0), (1.0, 3.0))`
		`println(m.t)`
		`println(m * m)`
		`println(m \ DenseVector(3.0, 5.0))`

10	Plotly.Scala imports	`import plotly._, plotly.element._, plotly.layout._`
11	Bar trace	`val barTrace = Bar(fruits.toSeq, revenues.toArray.toSeq, name = "Revenue (INR)")`

12	Layout with title and axis titles	`val plotLayout = Layout(`
		`  title = "Total Revenue by Fruit (Breeze & Plotly.Scala)",`
		`  xaxis = plotly.layout.Axis(title = "Fruit Variety"),`
		`  yaxis = plotly.layout.Axis(title = "Total Revenue (INR)"))`

13	Write the interactive HTML chart (also opens it in the browser)	`Plotly.plot("fruit_revenue.html", Seq(barTrace), plotLayout)`

Note. Breeze 2.1: * and *:* between two DenseVectors are both element-wise (the old :* operator no longer exists); the inner product is dot. variance/stddev are the sample versions (n-1). Plotly.plot also opens the chart in the browser unless openInBrowser = false.

S5. Spire exact arithmetic with a hand-made statistic (Program 25)

Lab records: Program 25. 7 rows; the rows form one program, executed top to bottom.

T.No	Task Desc	Program code
1	Import Spire	`import spire.math._` `import spire.implicits._`

2	Entry point	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

3	Price lists	`val prices = Map("Apple" -> List(120.50, 125.00, 118.75, 130.25, 128.00),` `  "Banana" -> List(44.00, 48.00, 45.00, 49.00, 45.00))`

4	Exact rational numbers	`val r = Rational(1, 10) + Rational(2, 10)`
		`println(r)`
		`println(r == Rational(3, 10))`
		`println(0.1 + 0.2)`

5	Exact sum and mean of a price list	`val pList = prices("Apple")`
		`val n = pList.length`
		`val spireSum = pList.map(Rational(_)).qsum`
		`val spireMean = spireSum / n`
		`println(spireMean)`
		`println(spireMean.toDouble)`

6	Sample variance and standard deviation	`val variance = pList.map(p => math.pow(p - spireMean.toDouble, 2)).sum / (n - 1)`
		`val stdDev = math.sqrt(variance)`
		`println(f"Variance: $variance%.4f  Std dev: $stdDev%.4f")`

7	Table row per fruit	`for ((f, ps) <- prices) {`
		`  val m = ps.map(Rational(_)).qsum / ps.length`
		`  println(f"  $f%-8s ${ps.length}%2d ${m.toDouble}%10.4f")`
		`}`

Note. Rational(1/3) is Rational(0) because 1/3 is integer division; write Rational(1, 3).

Part 2. Debug the program (S.No | code | description)

Each row is a small program with one bug. The description column shows what Python or the Scala compiler really printed, why it happens and the corrected code. Items marked **No error, wrong result** are the dangerous ones: nothing crashes, so you have to spot the mistake by reading the code. Every buggy snippet and every fix was run; the messages are copied from those runs.

2.1 Python: pandas (23 items)

Assumed before every item below:

```
import pandas as pd, numpy as np
df = pd.DataFrame({'Name': ['Asha', 'Ravi', 'Meena', 'John'], 'Age': [28, 35, 41, 30],
                   'City': ['Pune', 'Delhi', 'Pune', 'Delhi'], 'Salary': [52000, 61000, 75000, 48000]})
```

S.No	Code (buggy)	Description

1	`print(df.head)`	**No error, wrong result**
		`<bound method NDFrame.head of Name Age City Salary`
		`0 Asha 28 Pune 52000`
		`1 Ravi 35 Delhi 61000`
		A method was named but not called, so pandas prints the method object instead of the first rows. Add ( ).
		Fix:
		`print(df.head())`

2	`adults = df[df['Age'] > 30 and df['Salary'] > 50000]`	**Error raised**
		`ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(), a.any() or a.all().`
		Python's and needs a single True/False, a Series has many. Use the element-wise & and wrap each condition in ( ).
		Fix:
		`adults = df[(df['Age'] > 30) & (df['Salary'] > 50000)]`
		`print(adults)`

3	`adults = df[df['Age'] > 30 & df['Salary'] > 50000]`	**Error raised**
		`ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(), a.any() or a.all().`
		& binds tighter than >, so Python reads df['Age'] > (30 & df['Salary']) > 50000. That is a chained comparison, which needs a single True/False, so the error is the same as with and. Bracket every condition.
		Fix:
		`adults = df[(df['Age'] > 30) & (df['Salary'] > 50000)]`
		`print(adults)`

4	`print(df.sort_values('Age', ascending='False'))`	**Error raised**
		`ValueError: For argument "ascending" expected type bool, received type str.`
		ascending takes the boolean False, not the string 'False' (a non-empty string would be truthy).
		Fix:
		`print(df.sort_values('Age', ascending=False))`

5	`df = df.drop('Age')`
	`print(df.columns)`
		Error raised
		`KeyError: "['Age'] not found in axis"`
		drop looks for the label in the rows (axis=0) by default. To remove a column use columns='Age' or axis=1.
		Fix:
		`df = df.drop(columns='Age')`
		`print(df.columns.tolist())`

6	`d = df.copy()`
	`d.loc[1, 'Age'] = None`
	`d.dropna()`
	`print(len(d))`
		No error, wrong result
		`4`
		dropna() returns a new DataFrame and leaves d untouched. Assign the result back (or use inplace=True). No error is shown.
		Fix:
		`d = df.copy()`
		`d.loc[1, 'Age'] = None`
		`d = d.dropna()`
		`print(len(d))`

7	`print(df['name'])`	**Error raised**
		`KeyError: 'name'`
		Column labels are case sensitive: the column is 'Name', not 'name'. Check df.columns first.
		Fix:
		`print(df['Name'])`

S.No	Code (buggy)	Description

8	`print(df.iloc[:, 'Age'])`	**Error raised**
		`ValueError: Location based indexing can only have [integer, integer slice (START point is INCLUDED, END point is EXCLUDED), listlike of integers, boolean array] types`
		iloc is position based and accepts integers only. Use loc (or df['Age']) for a label, or iloc[:, 1] for a position.
		Fix:
		`print(df.loc[:, 'Age'])`

9	`print(df.groupby('City').mean())`	**Error raised**
		`TypeError: agg function failed [how->mean,dtype->object]`
		mean() cannot average the text column Name. Select the numeric columns first (or pass numeric_only=True).
		Fix:
		`print(df.groupby('City')[['Age', 'Salary']].mean())`

10	`a = pd.DataFrame({'id': ['1', '2', '3'], 'x': [10, 20, 30]})`
	`b = pd.DataFrame({'id': [1, 2, 3], 'y': [5, 6, 7]})`
	`print(pd.merge(a, b, on='id'))`
		Error raised
		`ValueError: You are trying to merge on object and int64 columns for key 'id'. If you wish to proceed you should use pd.concat`
		The join key is text in one frame and int in the other. Make the dtypes equal before merging.
		Fix:
		`a = pd.DataFrame({'id': ['1', '2', '3'], 'x': [10, 20, 30]})`
		`b = pd.DataFrame({'id': [1, 2, 3], 'y': [5, 6, 7]})`
		`a['id'] = a['id'].astype(int)`
		`print(pd.merge(a, b, on='id'))`

11	`d = df.copy()`
	`d.loc[1, 'Age'] = None`
	`d['Age'] = d['Age'].fillna(d['Age'].mean)`
	`print(d['Age'].dtype)`
		No error, wrong result
		`object`
		mean without ( ) is the method object, and fillna happily stores it in the gap, turning the column into dtype object. No error, so check dtypes after cleaning.
		Fix:
		`d = df.copy()`
		`d.loc[1, 'Age'] = None`
		`d['Age'] = d['Age'].fillna(d['Age'].mean())`
		`print(d['Age'].dtype, d['Age'].tolist())`

12	`df = pd.read_csv('sales_data.csv')`	**Error raised**
		`FileNotFoundError: [Errno 2] No such file or directory: 'sales_data.csv'`
		The file is not in the working directory. Put it next to the script, give the full path (raw string r'C:\...') or check os.getcwd().
		Fix:
		`df = pd.read_csv('sales_data.csv')`
		`print(df.shape)`

13	`df['Date'] = ['2026-01-05', '2026-02-10', '2026-03-15', '2026-04-20']`
	`print(df['Date'].dt.year)`
		Error raised
		`AttributeError: Can only use .dt accessor with datetimelike values`
		The column holds strings. The .dt accessor only works on datetime64, so convert with pd.to_datetime first.
		Fix:
		`df['Date'] = pd.to_datetime(['2026-01-05', '2026-02-10', '2026-03-15', '2026-04-20'])`
		`print(df['Date'].dt.year.tolist())`

14	`print(pd.Series(['1,200', '850']).astype(float))`	**Error raised**
		`ValueError: could not convert string to float: '1,200'`
		float() cannot read the thousands separator. Strip it with str.replace before astype.
		Fix:
		`print(pd.Series(['1,200', '850']).str.replace(',', '').astype(float).tolist())`

15	`d = df.copy()`
	`d[d['Age'] > 30]['Salary'] = 0`
	`print(d['Salary'].tolist())`
		No error, wrong result
		`[52000, 61000, 75000, 48000]`
		`+ SettingWithCopyWarning (printed to the console)`
		Chained indexing assigns into a temporary copy, so d is unchanged. Use a single .loc[row_mask, 'col'] = value.
		Fix:
		`d = df.copy()`
		`d.loc[d['Age'] > 30, 'Salary'] = 0`
		`print(d['Salary'].tolist())`

16	`both = pd.concat(df, df)`	**Error raised**
		`TypeError: concat() takes 1 positional argument but 2 were given`
		concat takes ONE positional argument: a list of frames. ignore_index=True renumbers the rows 0..n-1.
		Fix:
		`both = pd.concat([df, df], ignore_index=True)`
		`print(len(both))`

S.No	Code (buggy)	Description

17	`d = df.copy()`
	`d.loc[1, 'Age'] = None`
	`print(d['Age'].astype(int))`
		Error raised
		`IntCastingNaNError: Cannot convert non-finite values (NA or inf) to integer`
		NaN is a float and has no int equivalent. Fill (or drop) the missing values before casting, or use the nullable 'Int64'.
		Fix:
		`d = df.copy()`
		`d.loc[1, 'Age'] = None`
		`print(d['Age'].fillna(d['Age'].median()).astype(int).tolist())`

18	`print(df.corr())`	**Error raised**
		`ValueError: could not convert string to float: 'Asha'`
		corr() tries to convert every column to float and fails on the text columns. Restrict it to numeric ones.
		Fix:
		`print(df.corr(numeric_only=True))`

19	`df = df.sort_values('Age',`
	`inplace=True)`
	`print(df.head())`
		Error raised
		`AttributeError: 'NoneType' object has no attribute 'head'`
		With inplace=True the method returns None, so df became None. Use either inplace=True (no assignment) or the assignment (no inplace).
		Fix:
		`df = df.sort_values('Age')`
		`print(df.head().shape)`

20	`print(df[df['City'] = 'Pune'])`	**Error raised**
		`SyntaxError: cannot assign to subscript here. Maybe you meant '==' instead of '='?`
		= assigns, == compares. A filter needs ==.
		Fix:
		`print(df[df['City'] == 'Pune'])`

21	`print(df.isnull.sum())`	**Error raised**
		`AttributeError: 'function' object has no attribute 'sum'`
		isnull is a method: call it with ( ) before chaining .sum().
		Fix:
		`print(df.isnull().sum().sum())`

22	`df.columns = ['A', 'B']`	**Error raised**
		`ValueError: Length mismatch: Expected axis has 4 elements, new values have 2 elements`
		The list must have one name per column (4 here). Use df.rename(columns={...}) to rename only some.
		Fix:
		`df.columns = ['A', 'B', 'C', 'D']`
		`print(df.columns.tolist())`

23	`df['Dept'] = ['HR', 'IT']`	**Error raised**
		`ValueError: Length of values (2) does not match length of index (4)`
		A new column needs exactly one value per row (or a single scalar that is broadcast).
		Fix:
		`df['Dept'] = ['HR', 'IT', 'HR', 'IT']`
		`print(df['Dept'].tolist())`

2.2 Python: NumPy (10 items)

Assumed before every item below:

```
import numpy as np
```

S.No	Code (buggy)	Description

1	`print(np.array([1, 2, 3]) +`
	`np.array([1, 2]))`
		Error raised
		`ValueError: operands could not be broadcast together with shapes (3,) (2,)`
		Shapes (3,) and (2,) cannot be broadcast. Broadcasting needs equal lengths or a length of 1 in a trailing dimension.
		Fix:
		`print(np.array([1, 2, 3]) + np.array([1, 2, 3]))`

2	`A = np.ones((2, 3))`
	`B = np.ones((2, 3))`
	`print(np.dot(A, B))`
		Error raised
		`ValueError: shapes (2,3) and (2,3) not aligned: 3 (dim 1) != 2 (dim 0)`
		Matrix product needs A.shape[1] == B.shape[0]. Transpose B so (2,3) x (3,2) works.
		Fix:
		`A = np.ones((2, 3))`
		`B = np.ones((2, 3))`
		`print(np.dot(A, B.T))`

S.No	Code (buggy)	Description

3	`a = np.arange(8)`
	`print(a.reshape(3, 3))`
		Error raised
		`ValueError: cannot reshape array of size 8 into shape (3,3)`
		The new shape must hold exactly as many elements (8) as the array. -1 lets numpy work out one dimension: reshape(2, -1).
		Fix:
		`a = np.arange(8)`
		`print(a.reshape(2, 4))`

4	`A = np.array([[1, 2], [3, 4]])`
	`B = np.array([[5, 6], [7, 8]])`
	`print(A * B)`
		No error, wrong result
		`[[ 5 12]`
		` [21 32]]`
		* multiplies element by element. The matrix product is @ (or np.dot / np.matmul). No error is raised.
		Fix:
		`A = np.array([[1, 2], [3, 4]])`
		`B = np.array([[5, 6], [7, 8]])`
		`print(A @ B)`

5	`A = np.arange(6).reshape(2, 3)`
	`print(A.mean(axis=2))`
		Error raised
		`AxisError: axis 2 is out of bounds for array of dimension 2`
		A 2-D array has axes 0 (rows) and 1 (columns) only. axis=0 gives column means, axis=1 gives row means.
		Fix:
		`A = np.arange(6).reshape(2, 3)`
		`print(A.mean(axis=1))`

6	`a = np.array([1, 2, 3])`
	`b = a`
	`b[0] = 99`
	`print(a)`
		No error, wrong result
		`[99  2  3]`
		b = a only creates a second name for the same array, so editing b edits a. Slices are views too. Use .copy() for independent data.
		Fix:
		`a = np.array([1, 2, 3])`
		`b = a.copy()`
		`b[0] = 99`
		`print(a)`

7	`print(np.zeros(3, 3))`	**Error raised**
		`TypeError: Cannot interpret '3' as a data type`
		The shape is ONE argument: a tuple. The second positional argument of np.zeros is dtype.
		Fix:
		`print(np.zeros((3, 3)).shape)`

8	`M = np.array([[1, 2], [2, 4]])`
	`print(np.linalg.inv(M))`
		Error raised
		`LinAlgError: Singular matrix`
		The second row is twice the first, so det = 0 and the matrix has no inverse. Check np.linalg.det first or use the pseudo-inverse pinv.
		Fix:
		`M = np.array([[1, 2], [2, 4]])`
		`print(np.linalg.det(M))`
		`print(np.linalg.pinv(M).round(2))`

9	`a = np.array([1, 2, 'x'])`
	`print(a.mean())`
		Error raised
		`UFuncTypeError: ufunc 'add' did not contain a loop with signature matching types (dtype('<U11'), dtype('<U11')) -> None`
		One string turns the whole array into dtype '<U21' (text), so arithmetic fails. Clean the data so that the dtype is numeric (check a.dtype).
		Fix:
		`a = np.array([1, 2, 3])`
		`print(a.mean())`

10	`a = np.array([1, 2, 3, 4, 5])`
	`print(a[(a > 1) and (a < 5)])`
		Error raised
		`ValueError: The truth value of an array with more than one element is ambiguous. Use a.any() or a.all()`
		and / or / not work on single truth values. For arrays use & | ~ with brackets around each condition.
		Fix:
		`a = np.array([1, 2, 3, 4, 5])`
		`print(a[(a > 1) & (a < 5)])`

2.3 Python: Matplotlib and seaborn (6 items)

Assumed before every item below:

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd, numpy as np
df = pd.DataFrame({'City': ['Pune', 'Delhi', 'Goa'], 'Age': [28, 35, 41], 'Salary': [52000, 61000, 75000]})
```

S.No	Code (buggy)	Description

1	`plt.plot([1, 2, 3], [4, 5, 6, 7])`	**Error raised**
		`ValueError: x and y must have same first dimension, but have shapes (3,) and (4,)`
		x and y must have the same length (3 vs 4).
		Fix:
		`plt.plot([1, 2, 3], [4, 5, 6])`

2	`sns.barplot(x='City', y='Salary')`	**Error raised**
		`ValueError: Could not interpret value 'City' for 'x'. Value is a string, but 'data' was not passed.`
		Column names are strings, so seaborn needs data=df to look them up. Without it it cannot interpret 'City'.
		Fix:
		`sns.barplot(data=df, x='City', y='Salary')`

3	`plt.title = 'Salary by city'`
	`plt.plot([1, 2], [3, 4])`
	`plt.title('Salary by city')`
		Error raised
		`TypeError: 'str' object is not callable`
		Using = replaced the function plt.title with a string, so calling it later fails. Call the function, do not assign to it.
		Fix:
		`plt.plot([1, 2], [3, 4])`
		`plt.title('Salary by city')`

4	`plt.scatter(df.Age, df.Salry)`	**Error raised**
		`AttributeError: 'DataFrame' object has no attribute 'Salry'`
		Misspelled column name (Salry). Attribute access df.Salary only works for exact names without spaces.
		Fix:
		`plt.scatter(df.Age, df.Salary)`

5	`plt.plot([1, 2, 3], [4, 5, 6])`
	`plt.legend()`
		Runs, but warns
		`UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.`
		legend() only lists artists that have a label=. Without one the legend is empty and matplotlib just warns.
		Fix:
		`plt.plot([1, 2, 3], [4, 5, 6], label='trend')`
		`plt.legend()`

6	`plt.subplot(2, 2, 5)`	**Error raised**
		`ValueError: num must be an integer with 1 <= num <= 4, not 5`
		A 2 x 2 grid has panels 1 to 4 only (numbered row by row from 1).
		Fix:
		`plt.subplot(2, 2, 4)`

2.4 Python: Beautiful Soup and requests (6 items)

Assumed before every item below:

```
from bs4 import BeautifulSoup
import requests
html = """<div class="quote"><span class="text">Be yourself.</span><small class="author">Oscar Wilde</small><a>no
    link</a></div>
<div class="quote"><span class="text">Stay hungry.</span><small class="author">Steve Jobs</small></div>"""
soup = BeautifulSoup(html, 'html.parser')
```

S.No	Code (buggy)	Description

1	`quotes = soup.find_all('div',`
	`class='quote')`
		Error raised
		`SyntaxError: invalid syntax`
		class is a reserved word in Python. BeautifulSoup uses class_ (with the underscore), or attrs={'class': 'quote'}.
		Fix:
		`quotes = soup.find_all('div', class_='quote')`
		`print(len(quotes))`

S.No	Code (buggy)	Description

2	`print(soup.find('span', class_='txt').text)`	**Error raised**
		`AttributeError: 'NoneType' object has no attribute 'text'`
		find returns None when nothing matches (wrong class name or JavaScript-rendered page). Test the result before using .text.
		Fix:
		`tag = soup.find('span', class_='text')`
		`print(tag.text if tag else 'not found')`

3	`print(soup.find_all('span', class_='text').text)`	**Error raised**
		`AttributeError: ResultSet object has no attribute "text". You're probably treating a list of elements like a single element. Did you call find_all() when you meant to call find()?`
		find_all returns a list-like ResultSet. Loop over it (or index it) and take .text of each tag.
		Fix:
		`print([s.text for s in soup.find_all('span', class_='text')])`

4	`for div in soup.find_all('div', class_='quote'):`
	`  print(div.find('a')['href'])`
		Error raised
		`KeyError: 'href'`
		Indexing a tag with ['href'] raises KeyError if the attribute is missing; .get('href') returns None instead. The second quote even has no <a>.
		Fix:
		`for div in soup.find_all('div', class_='quote'):`
		`  a = div.find('a')`
		`  print(a.get('href') if a else None)`

5	`print(soup.find_all('span', class_='text')[5].text)`	**Error raised**
		`IndexError: list index out of range`
		Only two matches exist, so index 5 is out of range. Check len() before indexing.
		Fix:
		`spans = soup.find_all('span', class_='text')`
		`print(spans[0].text if spans else 'none')`

6	`r = requests.get('quotes.toscrape.com')`	**Error raised**
		`MissingSchema: Invalid URL 'quotes.toscrape.com': No scheme supplied. Perhaps you meant https://quotes.toscrape.com?`
		The URL must start with http:// or https://. (Fix not run here: it needs a live network.)
		Fix (not run here):
		`r = requests.get('https://quotes.toscrape.com')  # needs internet`

2.5 Python: Plotly (4 items)

Assumed before every item below:

```
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
df = pd.DataFrame({'City': ['Pune', 'Delhi', 'Goa'], 'Sales': [120, 90, 150]})
```

S.No	Code (buggy)	Description

1	`fig = px.bar(df, x='City', y='Sale')`	**Error raised**
		`ValueError: Value of 'y' is not the name of a column in 'data_frame'. Expected one of ['City', 'Sales'] but received: Sale`
		The y column name does not exist in the DataFrame. The message lists the valid names.
		Fix:
		`fig = px.bar(df, x='City', y='Sales')`
		`print(len(fig.data))`

2	`fig = go.Figure(go.Bar(x=df['City'], y=df['Sales']))`
	`fig.update_layout(titel='Sales')`
		Error raised
		`ValueError: Invalid property specified for object of type plotly.graph_objs.Layout: 'titel'`
		`Did you mean "title"?`
		Misspelled layout property. Plotly validates property names and lists the valid ones.
		Fix:
		`fig = go.Figure(go.Bar(x=df['City'], y=df['Sales']))`
		`fig.update_layout(title='Sales')`
		`print(fig.layout.title.text)`

S.No	Code (buggy)	Description

3	`go.Bar(x=df['City'], y=df['Sales'], color='red')`	**Error raised**
		`ValueError: Invalid property specified for object of type plotly.graph_objs.Bar: 'color'`
		`Did you mean "zorder"?`
		graph_objects have no color= shortcut (that is plotly.express). Use marker_color or marker=dict(color=...).
		Fix:
		`bar = go.Bar(x=df['City'], y=df['Sales'], marker_color='red')`
		`print(bar.marker.color)`

4	`fig = px.bar(df, x='City', y='Sales')`
	`fig.write_image('sales.png')`
		Error raised
		`RuntimeError: Image export requires the Kaleido package, v1.0.0 or greater, which can be installed using pip:`
		`$ pip install --upgrade "kaleido>=1"`
		Static export needs the kaleido package. Install it with pip, or use fig.write_html('sales.html') which needs nothing extra.
		Fix (not run here):
		`# pip install -U kaleido (then the same call works)`
		`fig.write_image('sales.png')`

2.6 Python: Bokeh (3 items)

Assumed before every item below:

```
from bokeh.plotting import figure, output_file, show
from bokeh.models import ColumnDataSource
```

S.No	Code (buggy)	Description

1	`p = figure(plot_width=400, plot_height=300)`	**Error raised**
		`AttributeError: unexpected attribute 'plot_width' to figure, similar attributes are outer_width, width or min_width`
		plot_width / plot_height were removed in Bokeh 3. Use width and height.
		Fix:
		`p = figure(width=400, height=300)`
		`print(p.width)`

2	`p = figure(width=300, height=300)`
	`p.circle([1, 2, 3], [4, 5, 6], size=10)`
		Runs, but warns
		`BokehDeprecationWarning: 'circle() method with size value' was deprecated in Bokeh 3.4.0 and will be removed, use 'scatter(size=...) instead' instead.`
		In Bokeh 3.4+ circle(size=...) is deprecated. Use scatter(), which also accepts a marker= argument.
		Fix:
		`p = figure(width=300, height=300)`
		`p.scatter([1, 2, 3], [4, 5, 6], size=10)`
		`print(len(p.renderers))`

3	`src = ColumnDataSource(dict(x=[1, 2, 3], y=[4, 5]))`	**Runs, but warns**
		`BokehUserWarning: ColumnDataSource's columns must be of the same length. Current lengths: ('x', 3), ('y', 2)`
		All columns of a ColumnDataSource must have the same length; Bokeh only warns, then the plot misbehaves.
		Fix:
		`src = ColumnDataSource(dict(x=[1, 2, 3], y=[4, 5, 6]))`
		`print(len(src.data['x']))`

2.7 Python: scikit-learn: PCA and K-means (6 items)

Assumed before every item below:

```
import numpy as np, pandas as pd
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
iris = load_iris(as_frame=True)
X = iris.data
df_iris = iris.data.copy()
df_iris['species'] = iris.target_names[iris.target]
```

S.No	Code (buggy)	Description

1	`pca = PCA(n_components=5).fit(X)`	**Error raised**
		`ValueError: n_components=5 must be between 0 and min(n_samples, n_features)=4 with svd_solver='covariance_eigh'`
		Components cannot exceed min(n_samples, n_features); iris has 4 features.
		Fix:
		`pca = PCA(n_components=2).fit(X)`
		`print(pca.n_components_)`

S.No	Code (buggy)	Description

2	`pca = PCA(n_components=2).fit(X)`
	`print(pca.explained_variance_ratio_.round(4))`
		No error, wrong result
		`[0.9246 0.0531]`
		Not an error but a wrong analysis: PCA follows variance, so a large-scale feature dominates. Standardise first (PC1 falls from 92.46% to 72.96%).
		Fix:
		`Xs = StandardScaler().fit_transform(X)`
		`pca = PCA(n_components=2).fit(Xs)`
		`print(pca.explained_variance_ratio_.round(4))`

3	`pca = PCA(n_components=2)`
	`print(pca.explained_variance_ratio_)`
		Error raised
		`AttributeError: 'PCA' object has no attribute 'explained_variance_ratio_'`
		Attributes ending in _ exist only after fit(). Call fit or fit_transform first.
		Fix:
		`pca = PCA(n_components=2).fit(X)`
		`print(pca.explained_variance_ratio_.round(2))`

4	`scaled = StandardScaler().fit_transform(X)`
	`print(scaled.head())`
		Error raised
		`AttributeError: 'numpy.ndarray' object has no attribute 'head'`
		sklearn transformers return a NumPy array, which has no .head(). Wrap it in pd.DataFrame(..., columns=...).
		Fix:
		`scaled = pd.DataFrame(StandardScaler().fit_transform(X), columns=X.columns)`
		`print(scaled.head(2).shape)`

5	`pcs = PCA(n_components=2).fit_transform(X)`
	`print(pcs['PC1'])`
		Error raised
		`IndexError: only integers, slices (`:`), ellipsis (`...`), numpy.newaxis (`None`) and integer or boolean arrays are valid indices`
		fit_transform returns an ndarray, so a column name cannot index it. Create a DataFrame with named columns first.
		Fix:
		`pcs = pd.DataFrame(PCA(n_components=2).fit_transform(X), columns=['PC1', 'PC2'])`
		`print(pcs['PC1'].shape)`

6	`km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(df_iris)`	**Error raised**
		`ValueError: could not convert string to float: 'setosa'`
		The species column is text. Clustering uses numeric features only, so drop the label column first.
		Fix:
		`km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(df_iris.drop(columns='species'))`
		`print(sorted(np.bincount(km.labels_)))`

2.8 Python: Plain Python (4 items)

S.No	Code (buggy)	Description

1	`total = 5`
	`print('Total: ' + total)`
		Error raised
		`TypeError: can only concatenate str (not "int") to str`
		+ cannot join a str and an int. Convert with str() or use an f-string: f'Total: {total}'.
		Fix:
		`total = 5`
		`print('Total: ' + str(total))`

2	`for i in range(3):`
	`print(i)`
		Error raised
		`IndentationError: expected an indented block after 'for' statement on line 1`
		The loop body must be indented (4 spaces).
		Fix:
		`for i in range(3):`
		`  print(i)`

3	`nums = [4, 8, 6]`
	`print(mean(nums))`
		Error raised
		`NameError: name 'mean' is not defined`
		mean is not a built-in. Import statistics.mean, or compute sum / len. The error NameError means a name was never defined or imported.
		Fix:
		`nums = [4, 8, 6]`
		`print(sum(nums) / len(nums))`

4	`lst = [1, 2]`
	`lst.append(3, 4)`
		Error raised
		`TypeError: list.append() takes exactly one argument (2 given)`
		append adds one element. extend (or +) adds several.
		Fix:
		`lst = [1, 2]`
		`lst.extend([3, 4])`
		`print(lst)`

2.9 Scala (26 items)

Each snippet is the body of main() (classes are shown above the statements). Compiled and run with Scala 2.13.12.

S.No	Code (buggy)	Description

1	`val x = 5`
	`x = 10`
		Compile error
		`reassignment to val`
		val / var. A val is a constant. Declare it with var if it must change (prefer new vals in functional code).
		Fix:
		`var x = 5`
		`x = 10`
		`println(x)`

2	`val arr = Array(1, 2, 3)`
	`println(arr[0])`
		Compile error
		`value arr of type Array[Int] does not take type parameters.`
		arrays. Scala indexes with round brackets arr(i). Square brackets are for type parameters, e.g. Array[Int].
		Fix:
		`val arr = Array(1, 2, 3)`
		`println(arr(0))`

3	`val l = List(1, 2, 3)`
	`l(0) = 10`
		Compile error
		`value update is not a member of List[Int]`
		`did you mean updated?`
		List. List (and the default Map/Set) is immutable, there is no update. updated returns a new list. Use Array or ListBuffer when you really need in-place changes.
		Fix:
		`val l = List(1, 2, 3)`
		`val l2 = l.updated(0, 10)`
		`println(l2)`

4	`val s = Set(1, 2)`
	`s += 3`
		Compile error
		`value += is not a member of scala.collection.immutable.Set[Int]`
		`did you mean ++?`
		`Expression does not convert to assignment because receiver is not assignable.`
		Set. The default Set is immutable, so += would have to reassign the val. Use s + 3 to get a new set, or scala.collection.mutable.Set / a var.
		Fix:
		`val s = Set(1, 2)`
		`val s2 = s + 3`
		`println(s2)`

5	`val m = Map("a" -> 1)`
	`println(m("b"))`
		Compiles, then throws
		`java.util.NoSuchElementException: key not found: b`
		Map. m(key) throws when the key is absent. get returns an Option (None if absent) and getOrElse supplies a default.
		Fix:
		`val m = Map("a" -> 1)`
		`println(m.getOrElse("b", 0))`
		`println(m.get("b"))`

6	`case class P(name: String, age: Int)`
	`val p = P("Ann", 20)`
	`p.age = 21`
		Compile error
		`reassignment to val`
		case class. Case-class parameters are immutable vals. copy(...) builds a new object with the changed field.
		Fix:
		`val p = P("Ann", 20)`
		`val p2 = p.copy(age = 21)`
		`println(p2)`

7	`val t = (1, "a")`
	`println(t(0))`
		Compile error
		`(Int, String) does not take parameters`
		tuples. Tuple elements are read with t._1, t._2 (counting from 1), or by destructuring. Tuples cannot be indexed with ( ).
		Fix:
		`val t = (1, "a")`
		`println(t._1)`
		`val (num, txt) = t`

8	`val x: Int = "5"`	**Compile error**
		`type mismatch;`
		`found : String("5")`
		`required: Int`
		types. Scala is statically typed: a String is not an Int. Convert explicitly with toInt / toDouble.
		Fix:
		`val x: Int = "5".toInt`
		`println(x + 1)`

S.No	Code (buggy)	Description

9	`val nums = List(1, 2, 3, 4)`
	`println(nums.sum / nums.length)`
		No error, wrong result
		`2`
		integer division. Int / Int is integer division (10 / 4 = 2). No error, just a wrong mean. Convert one operand to Double first.
		Fix:
		`val nums = List(1, 2, 3, 4)`
		`println(nums.sum.toDouble / nums.length)`

10	`val a = Array(1, 2, 3)`
	`println(a.length())`
		Compile error
		`Int does not take parameters`
		arrays. length is a parameterless member, so it is written without ( ). (size works as well.)
		Fix:
		`val a = Array(1, 2, 3)`
		`println(a.length)`

11	`val lst = List(1, 2, 3)`
	`println(s"Length: $lst.length")`
		No error, wrong result
		`Length: List(1, 2, 3).length`
		string interpolation. $name only takes the identifier. A method call or expression needs braces: ${lst.length}.
		Fix:
		`val lst = List(1, 2, 3)`
		`println(s"Length: ${lst.length}")`

12	`val name = "Rohan"`
	`println("Hello $name")`
		No error, wrong result
		`Hello $name`
		string interpolation. Without the s prefix the dollar sign is just text. f"..." (formatted) and raw"..." are the other interpolators.
		Fix:
		`val name = "Rohan"`
		`println(s"Hello $name")`

13	`val m = Map("a" -> 1)`
	`println(m.get("a") + 1)`
		Compile error
		`type mismatch;`
		`found   : Int(1)`
		`required: String`
		Option. get returns Option[Int], not Int, so + 1 is not defined. Unwrap with getOrElse, map or a pattern match.
		Fix:
		`val m = Map("a" -> 1)`
		`println(m.getOrElse("a", 0) + 1)`

14	`val g = mutable.HashMap("a" -> 1)`	**Compile error**
		`not found: value mutable`
		imports. mutable is not in scope. import scala.collection.mutable (the default collections are the immutable ones).
		Fix:
		`import scala.collection.mutable`
		`val g = mutable.HashMap("a" -> 1)`
		`g("b") = 2`
		`println(g.size)`

15	`import breeze.linalg._`
	`val v = DenseVector(1.0, 2.0) :*`
	`  DenseVector(3.0, 4.0)`
		Compile error
		`value :* is not a member of breeze.linalg.DenseVector[Double]`
		`did you mean *:*, :*=, or :=?`
		Breeze. The old element-wise operator :* is gone in Breeze 2.x. Use *:* (a plain * between two vectors is element-wise too) and dot for the inner product.
		Fix:
		`import breeze.linalg._`
		`val v = DenseVector(1.0, 2.0) *:* DenseVector(3.0, 4.0)`
		`println(v)`
		`println(DenseVector(1.0, 2.0) dot DenseVector(3.0, 4.0))`

16	`import spire.math._`
	`import spire.implicits._`
	`println(Rational(1 / 3))`
		No error, wrong result
		`0`
		Spire. 1 / 3 is integer division and is 0 before Rational ever sees it. Give the numerator and denominator separately.
		Fix:
		`import spire.math._`
		`import spire.implicits._`
		`println(Rational(1, 3))`
		`println(Rational(1, 3) + Rational(1, 6))`

S.No	Code (buggy)	Description

17	`import org.saddle._`
	`val s = Series(Vec(1.0, 2.0, 3.0),`
	`  Index("a", "b", "c"))`
	`println(s.mean)`
		Compile error
		`value mean is not a member of org.saddle.Series[String,Double]`
		`did you mean head or median?`
		Saddle. A Series has no mean method. Take the underlying Vec with .values (or .toVec) and call the statistic there.
		Fix:
		`import org.saddle._`
		`val s = Series(Vec(1.0, 2.0, 3.0), Index("a", "b", "c"))`
		`println(s.values.mean)`

18	`class A { def g: Int = 1 }`
	`class B extends A { def g: Int = 2 }`
	`val b = new B`
	`println(b.g)`
		Compile error
		`` `override` modifier required to override concrete member: ``
		`def g: Int (defined in class A)`
		OOP. Overriding a concrete member needs the override modifier.
		Fix:
		`class A { def g: Int = 1 }`
		`class B extends A { override def g: Int = 2 }`
		`val b = new B`
		`println(b.g)`

19	`class Emp(private val salary: Int)`
	`val e = new Emp(500)`
	`println(e.salary)`
		Compile error
		`value salary in class Emp cannot be accessed as a member of Emp from object`
		`  Experiment`
		OOP. A private member is visible only inside the class. Expose it through a public method, or drop private.
		Fix:
		`class Emp(private val salary: Int) { def getSalary: Int = salary }`
		`val e = new Emp(500)`
		`println(e.getSalary)`

20	`class Foo(val x: Int)`
	`val f = Foo(3)`
	`println(f.x)`
		Compile error
		`not found: value Foo`
		OOP. Only case classes get an apply method, so a plain class needs new. (Scala 3 relaxes this, Scala 2.13 does not.)
		Fix:
		`val f = new Foo(3)`
		`println(f.x)`

21	`println("abc".toInt)`	**Compiles, then throws**
		`java.lang.NumberFormatException: For input string: "abc"`
		conversion. toInt throws on text that is not a number. toIntOption returns None instead, which you can default with getOrElse.
		Fix:
		`println("abc".toIntOption)`
		`println("42".toIntOption.getOrElse(0))`

22	`val a = Array(1, 2, 3)`
	`println(a)`
		No error, wrong result
		`[I@4e7dc304`
		arrays. Arrays are JVM arrays, so println shows the type code and hash ([I@...). Use mkString or toList to see the elements.
		Fix:
		`val a = Array(1, 2, 3)`
		`println(a.mkString(", "))`
		`println(a.toList)`

23	`val l = List(1, 2, 3)`
	`l :+ 4`
	`println(l)`
		No error, wrong result
		`List(1, 2, 3)`
		List. Immutable collections never change in place: :+ returns a NEW list, which was thrown away here. Keep the result in a val (or use a var / ListBuffer).
		Fix:
		`val l = List(1, 2, 3)`
		`val l2 = l :+ 4`
		`println(l2)`

24	`val a = Array(10, 20, 30)`
	`for (i <- 0 to a.length) println(a(i))`
		Compiles, then throws
		`java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3`
		ranges. to includes the end value, until excludes it. Indices are 0 .. length-1, so use until (or a.indices).
		Fix:
		`val a = Array(10, 20, 30)`
		`for (i <- 0 until a.length) println(a(i))`

25	`def square(x: Int) { x * x }`
	`println(square(3))`
		No error, wrong result
		`()`
		methods. Without = the body is a procedure that returns Unit, so the value is lost and () is printed. Always write = after the parameter list.
		Fix:
		`def square(x: Int): Int = x * x`
		`println(square(3))`

S.No	Code (buggy)	Description

26	`println(List(1, 2, 3).get(0))`	**Compile error**
		`value get is not a member of List[Int]`
		List. List has no get (that is Java, and Map.get). Use list(i), head or headOption.
		Fix:
		`println(List(1, 2, 3)(0))`
		`println(List(1, 2, 3).headOption)`

Part 3. Fill in the missing lines (Line No | Missing code | Justification)

Each program below is complete in the answer table and was run end to end. Cover the answer table, fill the red lines yourself, then check. A good justification names what the line does **and** what would go wrong without it (wrong type, wrong axis, no output file).

F1. Employee table: create, filter, group, sort (pandas basics)

Lab records: Programs 1, 2. ***Task:*** *Build a DataFrame of employees, add a bonus column, filter the IT department, find the average salary per department and list the top three earners.* ***Fill lines 7, 11, 12, 14, 16.***

```
1  import pandas as pd
2
3  data = {"Name": ["Asha", "Ravi", "Meena", "John", "Sara", "Kabir"],
4          "Dept": ["HR", "IT", "IT", "HR", "Sales", "IT"],
5          "Age": [28, 35, 41, 30, 26, 38],
6          "Salary": [52000, 61000, 75000, 48000, 43000, 68000]}
7  ________ line 7: missing ________
8  print(df.shape)
9  print(df["Salary"].describe().round(1))
10
11 ________ line 11: missing ________
12 ________ line 12: missing ________
13 print(len(it), "IT employees")
14 ________ line 14: missing ________
15 print(avg.round(0))
16 ________ line 16: missing ________
17 print(top[["Name", "Salary"]])
```

Line No	Missing code	Justification
7	`df = pd.DataFrame(data)`	pd.DataFrame turns the dict into a table: each key becomes a column, each list the column values.
11	`df["Bonus"] = df["Salary"] * 0.10`	Assigning to a new column label creates it. The multiplication is vectorised, so no loop is needed.
12	`it = df[df["Dept"] == "IT"]`	A boolean mask inside [ ] keeps the rows where the condition is True. == compares, a single = would be a syntax error.
14	`avg = df.groupby("Dept")["Salary"].mean()`	groupby splits by department, selecting ["Salary"] limits the work to one column, mean() aggregates each group.
16	`top = df.sort_values("Salary", ascending=False).head(3)`	sort_values orders by Salary, ascending=False (a boolean, not a string) puts the highest first, head(3) keeps three rows.

Output of the complete program:

```
(6, 4)
count        6.0
mean     57833.3
std      12319.4
min      43000.0
25%      49000.0
50%      56500.0
75%      66250.0
max      75000.0
... (output continues)
```

F2. Cleaning a dirty table: duplicates, NaN, text, dates, outliers

Lab records: Programs 3, 4. ***Task:*** *Remove duplicate rows, fill the missing age with the median, tidy the city text, convert the date column and drop the outlier with the IQR rule.* ***Fill lines 10, 11, 12, 13, 16.***

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"id": [1, 2, 2, 3, 4, 5, 6],
                   "age": [25, 27, 27, np.nan, 220, 29, 26],
                   "city": ["Pune", "Delhi", "Delhi", None, "Goa", "pune ", "Delhi"],
                   "joined": ["2026-01-05", "2026-02-10", "2026-02-10", "2026-03-15",
                              "2026-04-20", "2026-05-25", "2026-06-30"]})
print(df.isnull().sum().to_dict())
________ line 10: missing ________
________ line 11: missing ________
________ line 12: missing ________
________ line 13: missing ________
q1, q3 = df["age"].quantile([0.25, 0.75])
iqr = q3 - q1
________ line 16: missing ________
print(clean[["id", "age", "city"]])
print(clean["joined"].dt.month.tolist())
```

Line No	Missing code	Justification
10	`df = df.drop_duplicates()`	Rows 2 and 3 are identical, drop_duplicates keeps the first copy. The result must be assigned back because it returns a new frame.
11	`df["age"] = df["age"].fillna(df["age"].median())`	fillna replaces NaN. The median is preferred to the mean here because the outlier 220 would distort the mean.
12	`df["city"] = df["city"].fillna("Unknown").str.strip().str.title()`	Fill the missing value first, then .str.strip() removes the stray space and .str.title() fixes the case ('pune ' becomes 'Pune').
13	`df["joined"] = pd.to_datetime(df["joined"])`	The column holds strings. to_datetime converts it to datetime64 so that the .dt accessor works.
16	`clean = df[df["age"].between(q1 - 1.5 * iqr, q3 + 1.5 * iqr)]`	IQR rule: keep values within 1.5 x IQR of the quartiles. between() builds the boolean mask, the outlier 220 is dropped.

Output of the complete program:

```
{'id': 0, 'age': 1, 'city': 1, 'joined': 0}
   id   age     city
0   1  25.0     Pune
1   2  27.0    Delhi
3   3  27.0  Unknown
5   5  29.0     Pune
6   6  26.0    Delhi
[1, 2, 3, 5, 6]
```

F3. Reshape, slice and broadcast an array (NumPy)

Lab records: Programs 5, 6. ***Task:*** *Create the numbers 1 to 12, reshape them to 3 x 4, compute column statistics, centre the columns and multiply by a matrix.* ***Fill lines 3, 4, 7, 8, 10, 14.***

```
1  import numpy as np
2
3  ________ line 3: missing ________
4  ________ line 4: missing ________
5  print(m.shape, m.ndim)
6  print(m[1, 2], m[:, 1], m[0:2, 1:3].tolist())
7  ________ line 7: missing ________
8  ________ line 8: missing ________
9  print(col_mean, row_sum)
10 ________ line 10: missing ________
11 print(centered[0])
12 print(m[m % 2 == 0])
13 w = np.array([[1, 0], [0, 1], [1, 1], [0, 2]])
14 ________ line 14: missing ________
15 print(prod)
```

Line No	Missing code	Justification
3	`a = np.arange(1, 13)`	arange(start, stop) excludes the stop value, so 1..12 needs stop = 13.
4	`m = a.reshape(3, 4)`	reshape needs 3 x 4 = 12 elements, the same count as a. It returns a view of the same data.
7	`col_mean = m.mean(axis=0)`	axis=0 runs down the rows, so the result has one mean per column (4 values).
8	`row_sum = m.sum(axis=1)`	axis=1 runs along each row, giving one total per row (3 values).
10	`centered = m - col_mean`	Broadcasting: the (4,) vector is stretched across the 3 rows of the (3, 4) matrix, so every column is centred.
14	`prod = m @ w`	@ is the matrix product. The inner dimensions must match: (3, 4) @ (4, 2) gives (3, 2). * would be element-wise and fail.

Output of the complete program:

```
(3, 4) 2
7 [ 2  6 10] [[2, 3], [6, 7]]
[5. 6. 7. 8.] [10 26 42]
[-4. -4. -4. -4.]
[ 2  4  6  8 10 12]
[[ 4 13]
 [12 29]
 [20 45]]
```

F4. Distribution, scatter and correlation heat-map (matplotlib and seaborn)

Lab records: Programs 7, 8. ***Task:*** *Create study-hours and score data, print their correlation and draw three panels: histogram, scatter plot and correlation heat-map, then save the figure.* ***Fill lines 11, 12, 14, 16, 18.***

```
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  rng = np.random.default_rng(42)
7  df = pd.DataFrame({"hours": rng.uniform(1, 10, 60)})
8  df["score"] = 40 + 5 * df["hours"] + rng.normal(0, 4, 60)
9  print(df.corr().loc["hours", "score"].round(2))
10
11 ________ line 11: missing ________
12 ________ line 12: missing ________
13 ax[0].set_title("Score distribution")
14 ________ line 14: missing ________
15 ax[1].set_title("Hours vs score")
16 ________ line 16: missing ________
17 plt.tight_layout()
18 ________ line 18: missing ________
19 print("saved eda.png")
```

Line No	Missing code	Justification
11	`fig, ax = plt.subplots(1, 3, figsize=(12, 3.5))`	subplots(rows, cols) returns the figure and an array of axes, one per panel, so ax[0], ax[1], ax[2] exist.
12	`ax[0].hist(df["score"], bins=8, color="steelblue", edgecolor="white")`	hist bins the values into 8 intervals and draws the bar heights: the distribution of one numeric column.

Line No	Missing code	Justification
14	`sns.scatterplot(data=df, x="hours", y="score", ax=ax[1])`	data=df lets seaborn look the column names up. ax=ax[1] draws into the second panel.
16	`sns.heatmap(df.corr(), annot=True, cmap="Blues", ax=ax[2])`	heatmap colours the correlation matrix, annot=True writes the numbers in the cells.
18	`plt.savefig("eda.png", dpi=100)`	savefig writes the figure to a file. It must come before plt.show(), which clears the figure.

Output of the complete program:

```
0.97
saved eda.png
```

F5. Scraping job cards with BeautifulSoup

Lab records: Programs 9, 10. ***Task:*** *Parse the HTML, loop over every job card, extract title, location and link and put the result in a DataFrame.* ***Fill lines 7, 9, 10, 12, 14.***

```
1  from bs4 import BeautifulSoup
2  import pandas as pd
3
4  html = """<div class="job"><h2>Data Analyst</h2><span class="loc">Pune</span><a href="/j/1">Apply</a></div>
5  <div class="job"><h2>ML Engineer</h2><span class="loc">Delhi</span><a href="/j/2">Apply</a></div>
6  <div class="job"><h2>BI Developer</h2><span class="loc">Pune</span><a href="/j/3">Apply</a></div>"""
7  ________ line 7: missing ________
8  rows = []
9  ________ line 9: missing ________
10     ________ line 10: missing ________
11     loc = card.find("span", class_="loc").get_text()
12     ________ line 12: missing ________
13     rows.append({"title": title, "location": loc, "link": link})
14 ________ line 14: missing ________
15 print(df)
16 print(df["location"].value_counts().to_dict())
```

Line No	Missing code	Justification
7	`soup = BeautifulSoup(html, "html.parser")`	Builds the parse tree from the HTML text. "html.parser" is the parser that ships with Python (no extra install).
9	`for card in soup.find_all("div", class_="job"):`	find_all returns every matching tag. class_ (with underscore) because class is a reserved word in Python.
10	`title = card.find("h2").text.strip()`	find returns the first match inside this card. .text gives the inner text and .strip() removes surrounding whitespace.
12	`link = card.find("a")["href"]`	An attribute is read like a dictionary key. Use .get("href") instead if the attribute may be missing.
14	`df = pd.DataFrame(rows)`	A list of dictionaries becomes a table: one row per dict, the keys become the column names.

Output of the complete program:

```
title location  link
0  Data Analyst     Pune  /j/1
1   ML Engineer    Delhi  /j/2
2  BI Developer     Pune  /j/3
{'Pune': 2, 'Delhi': 1}
```

F6. Interactive bar chart and OLS trend line (Plotly Express)

Lab records: Programs 11, 12. ***Task:*** *Compute revenue per fruit, draw a coloured bar chart with axis titles, save it as HTML, then fit an OLS trend line and print R-squared.* ***Fill lines 8, 9, 10, 11, 13.***

```
import pandas as pd
import plotly.express as px

df = pd.DataFrame({"fruit": ["Apple", "Banana", "Orange", "Mango", "Grape"],
                   "price": [120, 45, 80, 150, 95],
                   "qty": [250, 600, 320, 180, 210]})
df["revenue"] = df["price"] * df["qty"]
_________ line 8: missing _________
_________ line 9: missing _________
_________ line 10: missing _________
_________ line 11: missing _________
model = px.get_trendline_results(sc).iloc[0]["px_fit_results"]
_________ line 13: missing _________
print(len(fig.data), len(sc.data))
```

Line No	Missing code	Justification
8	`fig = px.bar(df, x="fruit", y="revenue", color="fruit", title="Revenue by fruit")`	px.bar(data_frame, x, y): x and y are column names as strings, color maps a column to bar colours.
9	`fig.update_layout(xaxis_title="Fruit", yaxis_title="Revenue (INR)", showlegend=False)`	update_layout changes titles and legend after the figure exists. The property names are validated (a typo raises ValueError).
10	`fig.write_html("revenue.html")`	Saves a stand-alone interactive page. It needs no extra package, unlike write_image which needs kaleido.
11	`sc = px.scatter(df, x="price", y="qty", trendline="ols", text="fruit")`	trendline="ols" adds a least-squares line and requires statsmodels to be installed.
13	`print(round(model.rsquared, 3))`	get_trendline_results returns the fitted statsmodels model, .rsquared is the share of variance in qty explained by price.

Output of the complete program:

```
0.75
5 2
```

F7. Iris scatter with colours by species and hover (Bokeh)

Lab records: Programs 13, 14, 15. ***Task:*** *Plot sepal length against petal length, colour the points by species with factor_cmap, add a hover tool and save the plot to an HTML file.* ***Fill lines 10, 11, 13, 14, 15.***

```
import pandas as pd
from bokeh.plotting import figure, output_file, save
from bokeh.models import ColumnDataSource, HoverTool
from bokeh.transform import factor_cmap
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.data
df["species"] = iris.target_names[iris.target]
_________ line 10: missing _________
_________ line 11: missing _________
p = figure(width=500, height=400, title="Iris: sepal vs petal length")
_________ line 13: missing _________
_________ line 14: missing _________
_________ line 15: missing _________
save(p)
print(len(p.renderers), p.title.text)
```

Line No	Missing code	Justification
10	`src = ColumnDataSource(df)`	A ColumnDataSource is Bokeh's data container. Plots and tools that use the same source stay linked (shared selection, hover).
11	`colors = factor_cmap("species", palette=["#1f77b4", "#ff7f0e", "#2ca02c"], factors=list(iris.target_names))`	factor_cmap maps each category in the 'species' column to one palette colour: three factors, three colours.
13	`p.scatter("sepal length (cm)", "petal length (cm)", source=src, color=colors, legend_field="species", size=7)`	scatter draws the markers from source columns. legend_field builds one legend entry per species. (circle(size=...) is deprecated in Bokeh 3.4+.)
14	`p.add_tools(HoverTool(tooltips=[("species", "@species"), ("petal", "@{petal length (cm)}")]))`	@column reads a source value. Names that contain spaces or brackets need the @{...} form.
15	`output_file("iris.html")`	Chooses the HTML file that save() or show() writes. Without it, save() has no target file.

Output of the complete program:

```
1 Iris: sepal vs petal length
```

F8. PCA on the iris data: scale, fit, variance ratio, loadings

Lab records: Program 16. ***Task:*** *Standardise the four iris measurements, project them on two principal components, print the explained variance and the loadings.* ***Fill lines 9, 10, 11, 16.***

```
1  import pandas as pd
2  from sklearn.datasets import load_iris
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.decomposition import PCA
5  import plotly.express as px
6
7  iris = load_iris(as_frame=True)
8  X = iris.data
9  ________ line 9: missing ________
10 ________ line 10: missing ________
11 ________ line 11: missing ________
12 ratio = pca.explained_variance_ratio_
13 print(ratio.round(4), ratio.sum().round(4))
14 df = pd.DataFrame(pcs, columns=["PC1", "PC2"])
15 df["species"] = iris.target_names[iris.target]
16 ________ line 16: missing ________
17 print(loadings.round(2))
18 fig = px.scatter(df, x="PC1", y="PC2", color="species", title="Iris PCA")
19 print(fig.layout.title.text)
```

Line No	Missing code	Justification
9	`Xs = StandardScaler().fit_transform(X)`	PCA follows variance, so features must be on the same scale first. fit_transform learns the mean/std and applies them (returns an ndarray).
10	`pca = PCA(n_components=2)`	n_components must be at most min(n_samples, n_features) = 4. Two components allow a 2-D plot.
11	`pcs = pca.fit_transform(Xs)`	fit_transform learns the components and returns the projected data (150 x 2). Calling fit_transform on the scaled data, not on X.
16	`loadings = pd.DataFrame(pca.components_.T, index=X.columns, columns=["PC1", "PC2"])`	components_ has one row per component, so transposing gives one row per original feature: the loadings.

Output of the complete program:

```
[0.7296 0.2285] 0.9581
                    PC1   PC2
sepal length (cm)  0.52  0.38
sepal width (cm)  -0.27  0.92
petal length (cm)  0.58  0.02
petal width (cm)   0.56  0.07
Iris PCA
```

F9. K-Means on the iris data: elbow, fit, silhouette

Lab records: Program 17. ***Task:*** *Standardise the iris data, compute the inertia for k = 1 to 6 (elbow), fit k = 3 and report cluster sizes and the silhouette score.* ***Fill lines 11, 13, 14, 16, 17.***

```
1  import numpy as np
2  from sklearn.datasets import load_iris
3  from sklearn.preprocessing import StandardScaler
4  from sklearn.cluster import KMeans
5  from sklearn.metrics import silhouette_score
6
7  X = load_iris(as_frame=True).data
8  Xs = StandardScaler().fit_transform(X)
9  inertia = []
10 for k in range(1, 7):
11     ________ line 11: missing ________
12 print([round(i, 1) for i in inertia])
13 ________ line 13: missing ________
14 ________ line 14: missing ________
15 print(sorted(np.bincount(labels).tolist()))
16 ________ line 16: missing ________
17 ________ line 17: missing ________
18 print(round(sil, 3))
```

Line No	Missing code	Justification
11	`inertia.append(KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xs).inertia_)`	inertia_ is the within-cluster sum of squares. Plotting it against k gives the elbow curve for choosing k.
13	`km = KMeans(n_clusters=3, n_init=10, random_state=42)`	n_clusters=3 is the chosen k, random_state fixes the random start so that the result can be reproduced.
14	`labels = km.fit_predict(Xs)`	fit_predict fits the model and returns the cluster number (0, 1, 2) of every row.
16	`print(km.cluster_centers_.shape)`	cluster_centers_ is a (k, n_features) array: one centroid per cluster.
17	`sil = silhouette_score(Xs, labels)`	Silhouette compares within-cluster and nearest-cluster distance, from -1 to 1. Higher means better separated clusters.

Output of the complete program:

```
[600.0, 222.4, 139.8, 114.1, 90.9, 81.5]
[47, 50, 53]
(3, 4)
0.46
```

F10. Arrays, lists, sets and maps

Lab records: Programs 18, 19, 20. ***Task:*** *Sum an array, square a list, intersect two character sets, merge two grade maps keeping the higher grade and overwrite a key of a mutable HashMap.* ***Fill lines 6, 8, 12, 15, 18.***

```
1  import scala.collection.mutable
2
3  object Experiment {
4    def main(args: Array[String]): Unit = {
5      val arr = Array(2, 5, 9, 14, 20)
6      ________ line 6: missing ________
7      val primes = List(2, 3, 5, 7, 11)
8      ________ line 8: missing ________
9      println(squares)
10     val s1 = "hello world".toSet
11     val s2 = "world wide web".toSet
12     ________ line 12: missing ________
13     val m1 = Map("Rohan" -> 92, "Vedant" -> 88)
14     val m2 = Map("Rohan" -> 85, "Kalyan" -> 90)
15     ________ line 15: missing ________
16     println(merged.toSeq.sortBy(_._1))
17     val grades = mutable.HashMap("Alice" -> 85)
18     ________ line 18: missing ________
19     println(grades)
20   }
21 }
```

Line No	Missing code	Justification
6	`println(arr.sum)`	sum is defined on every numeric collection. arr.sum returns 50 without a loop.
8	`val squares = primes.map(x => x * x)`	map applies the function to each element and returns a NEW list: List(4, 9, 25, 49, 121).
12	`println((s1 & s2).toList.sorted.mkString)`	& is intersection (also written intersect). A Set has no order, so sort before printing.

Line No	Missing code	Justification
15	`val merged = (m1.keySet ++ m2.keySet).map { k => k -> Math.max(m1.getOrElse(k, 0), m2.getOrElse(k, 0)) }.toMap`	Union of the keys, then getOrElse(k, 0) avoids an exception for a missing key. The key -> value pairs are turned back into a Map with toMap.
18	`grades.put("Alice", 95)`	A duplicate key overwrites the old value (put returns the old one as an Option). This is the hash-collision rule for equal keys.

Output of the complete program:

```
50
List(4, 9, 25, 49, 121)
 delorw
List((Kalyan,90), (Rohan,92), (Vedant,88))
HashMap(Alice -> 95)
```

F11. Functional combinators and a case class

Lab records: Programs 21, 22. ***Task:*** *Filter, map and fold a list of integers, then filter, group and rank a list of Person objects.* ***Fill lines 6, 7, 8, 12, 13.***

```
1  case class Person(name: String, age: Int, city: String)
2
3  object Experiment {
4    def main(args: Array[String]): Unit = {
5      val nums = (1 to 10).toList
6      ________ line 6: missing ________
7      ________ line 7: missing ________
8      ________ line 8: missing ________
9      println(s"$evens $tripled $product")
10     val people = List(Person("Vedant", 24, "Mumbai"), Person("Ayush", 32, "Delhi"),
11       Person("Priya", 35, "Mumbai"), Person("Sneha", 22, "Hyderabad"))
12     ________ line 12: missing ________
13     ________ line 13: missing ________
14     byCity.toSeq.sortBy(_._1).foreach { case (c, g) => println(s"$c: ${g.map(_.name).mkString(", ")}") }
15     val ranked = people.sortBy(-_.age)
16     ranked.zipWithIndex.foreach { case (p, i) => println(s"${i + 1}. ${p.name} (${p.age})") }
17     println(older.map(_.name))
18   }
19 }
```

Line No	Missing code	Justification
6	`val evens = nums.filter(_ % 2 == 0)`	filter keeps the elements for which the predicate is true. Underscore stands for the element.
7	`val tripled = nums.map(_ * 3)`	map transforms every element and keeps the list length.
8	`val product = nums.foldLeft(1)(_ * _)`	foldLeft(start)(op) combines left to right. The start value is 1 because it is the identity of multiplication (0 would give 0).
12	`val older = people.filter(_.age > 30)`	Case-class fields are public vals, so _.age works. The result is a List[Person].
13	`val byCity = people.groupBy(_.city)`	groupBy returns a Map from the key (city) to the list of people with that key.

Output of the complete program:

```
List(2, 4, 6, 8, 10) List(3, 6, 9, 12, 15, 18, 21, 24, 27, 30) 3628800
Delhi: Ayush
Hyderabad: Sneha
Mumbai: Vedant, Priya
1. Priya (35)
2. Ayush (32)
3. Vedant (24)
4. Sneha (22)
List(Ayush, Priya)
```

F12. Vectors, statistics and formatted output (Breeze)

Lab records: Program 24. ***Task:*** *Compute the revenue of each fruit as price x quantity, then its total and mean, print a formatted line per fruit and the inner product.* ***Fill lines 9, 10, 11, 12, 14.***

```
1  import breeze.linalg._
2  import breeze.stats._
3
4  object Experiment {
5    def main(args: Array[String]): Unit = {
6      val fruits = Array("Apple", "Banana", "Orange")
7      val prices = DenseVector(120.0, 45.0, 80.0)
8      val qty = DenseVector(250.0, 600.0, 320.0)
9      ________ line 9: missing ________
10     ________ line 10: missing ________
11     ________ line 11: missing ________
12     ________ line 12: missing ________
13     println(f"Total: $total%,.2f  Mean: $avg%.2f")
14     ________ line 14: missing ________
15   }
16 }
```

Line No	Missing code	Justification
9	`val revenue = prices *:* qty`	*:* multiplies element by element (price i x quantity i). The result is again a DenseVector of length 3.
10	`val total = sum(revenue)`	sum from breeze.linalg adds all the elements and returns a Double.
11	`val avg = mean(revenue)`	mean comes from breeze.stats, hence the second import.
12	`for (i <- fruits.indices)` `  println(f"${fruits(i)}%-7s ${revenue(i)}%,10.2f")`	The f interpolator formats: %-7s left-aligns the text in 7 characters, %,10.2f adds thousands separators with 2 decimals.
14	`println(prices dot qty)`	dot is the inner product (sum of products), a single number, not a vector.

Output of the complete program:

```
Apple    30,000.00
Banana   27,000.00
Orange   25,600.00
Total: 82,600.00  Mean: 27533.33
82600.0
```

F13. Series and Frame (Saddle)

Lab records: Program 23. ***Task:*** *Build a price Series indexed by date, print the mean, filter the expensive days, then build a two-column Frame and compute revenue.* ***Fill lines 8, 9, 10, 13, 15.***

```
1  import org.saddle._
2  import org.saddle.ops.BinOps._
3
4  object Experiment {
5    def main(args: Array[String]): Unit = {
6      val prices = Vec(120.50, 125.00, 118.75, 130.25, 128.00)
7      val dates = Index("2026-05-01", "2026-05-08", "2026-05-15", "2026-05-22", "2026-05-29")
8      ________ line 8: missing ________
9      ________ line 9: missing ________
10     ________ line 10: missing ________
11     println(high.index.toVec)
12     val qty = Vec(45.0, 50.0, 42.0, 55.0, 48.0)
13     ________ line 13: missing ________
14     println(frame.numRows + " x " + frame.numCols)
15     ________ line 15: missing ________
16     println(revenue.sum)
17   }
18 }
```

Line No	Missing code	Justification
8	`val s = Series(prices, dates)`	Series(values, index): the data Vec first, the label Index second.
9	`println(f"Average: ${s.values.mean}%.2f")`	A Series has no mean method. Take the Vec of values (.values) and call mean there.
10	`val high = s.filter(_ > 125.0)`	filter keeps the entries whose VALUE satisfies the predicate, together with their index labels.
13	`val frame = Frame("Price" -> prices, "Qty" -> qty)`	Frame(name -> Vec, ...) builds a table of named columns, all of the same length.

Line No	Missing code	Justification
15	`val revenue = frame.firstCol("Price").toVec *` `  frame.firstCol("Qty").toVec`	firstCol picks a column by name, toVec exposes it as a Vec, and * multiplies element by element.

Output of the complete program:

```
Average: 124.50
[2 x 1]
2026-05-22
2026-05-29
5 x 2
29967.75
```

Part 4. Reference: Load-package rows and function index

4.1 Load package (the first row of almost every Q1 table)

Library	Import line	Install
pandas	`import pandas as pd`	pip install pandas
numpy	`import numpy as np`	pip install numpy
matplotlib	`import matplotlib.pyplot as plt`	pip install matplotlib
seaborn	`import seaborn as sns`	pip install seaborn
requests	`import requests`	pip install requests
Beautiful Soup	`from bs4 import BeautifulSoup`	pip install beautifulsoup4
Plotly Express	`import plotly.express as px`	pip install plotly
Plotly graph objects	`import plotly.graph_objects as go`	pip install plotly

Bokeh	`from bokeh.plotting import figure, show, output_file, save`
	`from bokeh.models import ColumnDataSource, HoverTool, LabelSet`
	`from bokeh.transform import factor_cmap`
		pip install bokeh

scikit-learn	`from sklearn.preprocessing import StandardScaler`
	`from sklearn.decomposition import PCA`
	`from sklearn.cluster import KMeans`
	`from sklearn.metrics import silhouette_score`
	`from sklearn.datasets import load_iris`
		pip install scikit-learn

statsmodels (for trendline='ols')	`# imported by plotly itself, only has to be installed`	pip install statsmodels

Scala library	Import line	scala-cli directive
Mutable collections	`import scala.collection.mutable`	`part of Scala`
Breeze	`import breeze.linalg._` `import breeze.stats._`	`//> using dep org.scalanlp::breeze:2.1.0`
Saddle (maintained fork)	`import org.saddle._` `import org.saddle.ops.BinOps._`	`//> using dep io.github.pityka::saddle-core:4.0.0-M14`
Spire	`import spire.math._` `import spire.implicits._`	`//> using dep org.typelevel::spire:0.18.0`
Plotly.Scala	`import plotly._, plotly.element._, plotly.layout._`	`//> using dep org.plotly-scala::plotly-render:0.8.5`

4.2 Function to package index

Written by hand from the rows that were executed (the lab PDFs' own function tables could not be extracted cleanly).

pandas (import pandas as pd)

Function	Package	What it does
`pd.DataFrame(data, columns=)`	pandas	Builds a table from a dict of lists, a list of lists or a list of dicts.
`df.head(n) / df.tail(n)`	pandas	First / last n rows (default 5).
`df.shape, size, ndim, dtypes, columns, index`	pandas	Attributes (no brackets): (rows, cols), element count, dimensions, column types, labels.
`df['A'], df[['A', 'B']]`	pandas	One column (Series) / several columns (DataFrame).
`df[df['Age'] > 25]`	pandas	Boolean filtering. Combine with & | ~ and brackets, not and / or.
`df.query('Age > 25')`	pandas	Row filter written as a string expression.
`df.loc[row, col], df.iloc[r, c]`	pandas	Label based / position based selection (iloc excludes the end of a slice).
`df.sort_values(by=, ascending=)`	pandas	Sorts rows. ascending takes True or False.
`df.groupby('C')['A'].sum()`	pandas	Split by category, aggregate one column (sum, mean, count, agg([...])).
`df.groupby('C').agg(Name=('col', 'func'))`	pandas	Named aggregation, e.g. Job_Count=('Job_Title', 'count').

Function	Package	What it does
`s.mean(), sum(), min(), max(), median(), std()`	pandas	Column statistics. NaN values are skipped.
`s.idxmax(), s.idxmin()`	pandas	Index label of the largest / smallest value.
`s.value_counts()`	pandas	Frequency of every distinct value.
`df.describe()`	pandas	count, mean, std, min, quartiles, max of numeric columns.
`df.corr()`	pandas	Correlation matrix of numeric columns.
`df.isnull(), notnull(), isnull().sum()`	pandas	Missing-value flags / count per column.
`df.fillna(value)`	pandas	Replaces NaN with a value (0, the mean, 'Unknown'). Returns a new frame.
`df.dropna(subset=[...])`	pandas	Drops rows with NaN (only in the listed columns if subset is given).
`df.drop(columns=[...])`	pandas	Removes columns (axis=1). Without columns= it looks for row labels.
`df.drop_duplicates()`	pandas	Removes repeated rows, keeps the first.
`df.reset_index(drop=True)`	pandas	Renumbers the rows 0..n-1 and discards the old index.
`df.rename(columns={'old': 'new'})`	pandas	Renames columns.
`s.astype(int)`	pandas	Converts the dtype. Fails on NaN: fill first.
`s.apply(func)`	pandas	Calls a Python function on every value.
`s.str.strip(), .title(), .replace()`	pandas	Vectorised text cleaning.
`pd.cut(s, bins=, labels=)`	pandas	Bins numbers into categories.
`pd.crosstab(a, b, margins=True)`	pandas	Contingency table of two categorical columns.
`df.to_csv(path, index=False), df.to_json(path, orient='records')`	pandas	Export to CSV / JSON.

numpy (import numpy as np)

Function	Package	What it does
`np.array([...]), np.arange(a, b, step), np.linspace(a, b, n)`	numpy	Create arrays. arange excludes the end, linspace includes it.
`np.zeros(shape), np.ones(shape), np.full(shape, v)`	numpy	Filled arrays. The shape is ONE tuple argument.
`np.random.seed(n), randint, rand, randn, uniform, normal`	numpy	Reproducible random numbers. randint(0, 11) gives 0..10.
`a.shape, a.size, a.dtype, a.ndim`	numpy	Attributes of an array.
`a.reshape(r, c)`	numpy	Changes the shape (same element count). -1 means 'work it out'.
`np.mean / np.std / np.sum (a, axis=)`	numpy	axis=0 per column, axis=1 per row. std is the population version unless ddof=1.
`np.argsort(a)[::-1]`	numpy	Positions that sort the array, reversed for highest first.
`np.where(cond, x, y)`	numpy	Element-wise if / else. With only cond it returns the positions.
`np.log(a), np.clip(a, lo, hi), np.polyfit(x, y, 1)`	numpy	Log transform, limit values, least-squares line (slope, intercept).
`a * b, a @ b, a.T`	numpy	Element-wise product, matrix product, transpose.
`a[:2, -2:]`	numpy	Slicing rows, columns: first two rows, last two columns.

matplotlib / seaborn

Function	Package	What it does
`plt.figure(figsize=), plt.subplots(r, c)`	matplotlib	New figure / grid of axes.
`plt.scatter, plt.bar, plt.hist, plt.plot`	matplotlib	Basic chart types.
`plt.xlabel, plt.ylabel, plt.title, plt.legend, plt.colorbar`	matplotlib	Labels and decorations. Do not assign to them (plt.title = 'x' breaks the function).
`plt.tight_layout(), plt.savefig(path), plt.show()`	matplotlib	Spacing, write to file (before show), display.
`sns.scatterplot(data=, x=, y=, hue=)`	seaborn	Scatter plot with colour groups. data=df is required for column names.
`sns.boxplot(data=, x=, y=)`	seaborn	Box plot of a value per category.
`sns.heatmap(corr, annot=True, cmap=)`	seaborn	Coloured matrix, numbers in the cells.

requests / Beautiful Soup

Function	Package	What it does
`requests.get(url, timeout=)`	requests	Downloads a page. resp.status_code == 200 means success, resp.text is the HTML.
`BeautifulSoup(html, 'html.parser')`	bs4	Parses HTML into a searchable tree.
`soup.find(tag, class_=), soup.find_all(...)`	bs4	First match (or None) / list of all matches. class_ has the underscore.
`soup.select('div.quote span.text')`	bs4	CSS selector search, returns a list.
`tag.text, tag.get_text(), tag['href'], tag.get('href')`	bs4	Inner text and attributes. get returns None if the attribute is missing.

Plotly Express / graph objects

Function	Package	What it does
`px.bar(df, x=, y=, color=, text=, title=)`	plotly.express	Interactive bar chart.
`px.scatter(df, x=, y=, size=, color=, trendline='ols')`	plotly.express	Scatter with optional least-squares line (needs statsmodels).
`px.scatter_3d(df, x=, y=, z=, color=)`	plotly.express	3-D scatter.
`fig.update_traces(...), fig.update_layout(...)`	plotly	Change the data series / the layout (titles, axes, background).
`fig.add_annotation(x=, y=, text=)`	plotly	Text label on the chart.
`fig.show(), fig.write_html(path), fig.write_image(path)`	plotly	Display / save page / save PNG (write_image needs kaleido).

Bokeh

Function	Package	What it does
`figure(title=, width=, height=, x_range=)`	bokeh.plotting	Canvas. A list of strings as x_range makes a categorical axis.
`ColumnDataSource(df)`	bokeh.models	Data container shared by glyphs and tools.
`p.scatter / p.line / p.vbar`	bokeh.plotting	Glyph methods. circle(size=) is deprecated in Bokeh 3.4+.
`factor_cmap(field, palette, factors)`	bokeh.transform	Category to colour mapping.
`HoverTool(tooltips=[(label, '@column')])`	bokeh.models	Tooltip on hover.
`LabelSet(x=, y=, text=, source=)`	bokeh.models	Text labels on the glyphs.
`output_file(path), show(p), save(p)`	bokeh.plotting	Choose the HTML file, open it, write it.
`p.legend.click_policy = 'hide'`	bokeh	Click a legend entry to hide the series.

scikit-learn

Function	Package	What it does
`load_iris()`	sklearn.datasets	150 flowers, 4 features, 3 species (.data, .target, .target_names).
`StandardScaler().fit_transform(X)`	sklearn.preprocessing	Mean 0, standard deviation 1 per column. inverse_transform undoes it.
`PCA(n_components=k).fit(X) / fit_transform(X)`	sklearn.decomposition	Principal components. Scale first; k <= min(n_samples, n_features).
`pca.explained_variance_ratio_, pca.components_`	sklearn.decomposition	Variance share per component / loadings (one row per component).
`KMeans(n_clusters=k, n_init=10, random_state=42)`	sklearn.cluster	K-means. fit_predict(X) returns the cluster labels.
`km.inertia_, km.cluster_centers_, km.labels_`	sklearn.cluster	Within-cluster sum of squares, centroids, labels (learned attributes end with _).
`silhouette_score(X, labels)`	sklearn.metrics	-1 to 1, higher means better separated clusters.

Scala collections and functions

Function	Package	What it does
`Array(1, 2, 3), arr(i), arr.length`	scala	Fixed size, mutable elements, round-bracket indexing.
`List(...), Set(...), Map(k -> v), (a, b, c)`	scala	Immutable list, set, map, tuple (t._1, t._2).
`mutable.HashMap, mutable.Map`	scala.collection.mutable	Mutable map: put, update with m(k) = v.
`m.get(k), m.getOrElse(k, d), m.contains(k)`	scala	Safe lookups (get returns an Option).
`map, filter, flatMap, foldLeft, reduce`	scala	Functional combinators.
`sum, exists, take, partition, groupBy, sortBy, zipWithIndex`	scala	More collection operations.
`s1 & s2, s1 | s2, s1 -- s2`	scala	Set intersection, union, difference.
`case class P(name: String, age: Int); p.copy(age = 25)`	scala	Immutable record with equals, toString and pattern matching.
`s"...$x ${x + 1}", f"$x%.2f"`	scala	String interpolation (s) and formatted output (f).

Breeze / Saddle / Spire / Plotly.Scala

Function	Package	What it does
`DenseVector(...), DenseMatrix((...), (...))`	breeze.linalg	Numeric vector / matrix.
`a *:* b, a dot b, a + b, a * 2.0`	breeze.linalg	Element-wise product, inner product, addition, scaling.
`sum(v), mean(v), stddev(v), variance(v)`	breeze.linalg / breeze.stats	Statistics. stddev and variance are the sample versions.
`m.t, m * m, m \ v`	breeze.linalg	Transpose, matrix product, solve a linear system.
`Vec(...), Index(...), Series(vec, index), Frame("A" -> vec, ...)`	org.saddle	Labelled data structures. Series(values, index).
`series.values.mean, frame.firstCol("A").toVec`	org.saddle	A Series has no mean: go through .values / .toVec.
`Rational(1, 3), .qsum, .toDouble`	spire.math	Exact fractions. Rational(1/3) is Rational(0) because 1/3 is integer division.
`Bar(x, y, name), Layout(title, xaxis, yaxis), Plotly.plot(path, traces, layout)`	plotly	Plotly.Scala bar chart. Plotly.plot opens the browser unless openInBrowser = false.

Part 5. What was verified and what was not

Verified by running

- Part 1: 124 of the 126 Python rows were executed group by group in order (the other 2 are marked 'not run'); the 5 Scala tables were compiled and run with scala-cli. Failures: 0.
- Part 2: 62 Python and 26 Scala bugs. Each buggy snippet was run to capture the real message, each fix was run too (2 fixes need internet or kaleido, so they are marked 'not run here').
- Part 3: the 13 complete programs ran without error and their output is shown.
- Versions used: pandas 2.3.3, NumPy 1.26.4, scikit-learn 1.7.2, Plotly 7.1.0, Bokeh 3.9.2, seaborn 0.13.2, Python 3.10; Scala 2.13.12 on Java 17 with Breeze 2.1.0, Spire 0.18.0, Plotly.Scala 0.8.5. Error texts change between versions, so yours may be worded a little differently.

Not verified, or only partly

- **Marks and layout of the three formats** come from reading one photo. I do not know your rubric.
- **The code is rewritten, not copied.** The text extracted from the lab PDFs wraps long strings and puts the closing braces of Scala *main* on the wrong page, so I wrote clean versions of the same tasks and ran them. I did not diff them against your records line by line.
- **Datasets:** small inline data replaces whatever files or web pages the labs loaded. Outputs of random data depend on the seed and the library version.
- **Not executed:** a live `requests.get` of a real website, `fig.write_image` (kaleido is not installed here, I captured the error text only), Bokeh `export_png` (needs a browser driver), and `show()` calls, which I patched so that no browser opened (the figures were built but I did not look at them).
- **Vegas:** programs 23 and 25 of the labs use Vegas, which has no Scala 2.13 build. The Saddle rows were verified with the maintained fork `io.github.pityka::saddle-core:4.0.0-M14`, so the API may differ slightly from the original `org.scala-saddle` your lab used.
- **Plotly.Scala:** `Plotly.plot` opens a browser by default. It was verified with `openInBrowser = false`.
- **Coverage:** only lab programs 1 to 25. Spark, regular expressions and the other lecture topics are not in this sheet; see the earlier question bank and topic notes for those.
- **Function index:** hand-written because the function tables in the lab PDFs do not extract cleanly.